

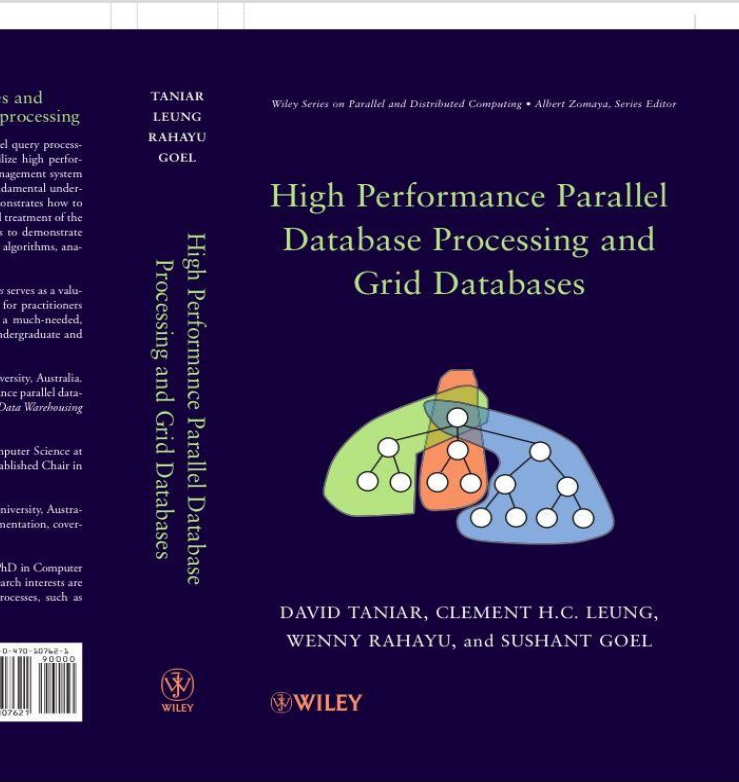
FIT5202 – Data Processing for Big Data

Week 12 Revision



Unit Overview

1. **Volume** □ Sessions 1, 2, 3, 4
 - How to process Big Data Volume?
2. **Complexity** □ Sessions 5, 6, 7, 8
 - How to apply machine learning algorithms to every aspect of Big Data?
3. **Velocity** □ Sessions 9, 10, 11
 - How to handle and process Fast Streaming Data?



Chapter 1 Introduction

- 1.1 A Brief Overview - Parallel Databases and Grid Databases
- 1.2 Parallel Query Processing: Motivations
- 1.3 Parallel Query Processing: Objectives
- 1.4 Forms of Parallelism
- 1.5 Parallel Database Architectures
- 1.6 Grid Database Architecture
- 1.7 Structure of this Book
- 1.8 Summary
- 1.9 Bibliographical Notes
- 1.10 Exercises

1.3. Objectives (cont'd)

• Speed up

- Performance improvement gained because of extra processing elements added
 - Running a given task in less time by increasing the degree of parallelism
-
- Linear speed up: performance improvement growing linearly with additional resources
 - Superlinear speed up
 - Sublinear speed up

$$\text{Speed up} = \frac{\text{elapsed time on uniprocessor}}{\text{elapsed time on multiprocessors}}$$

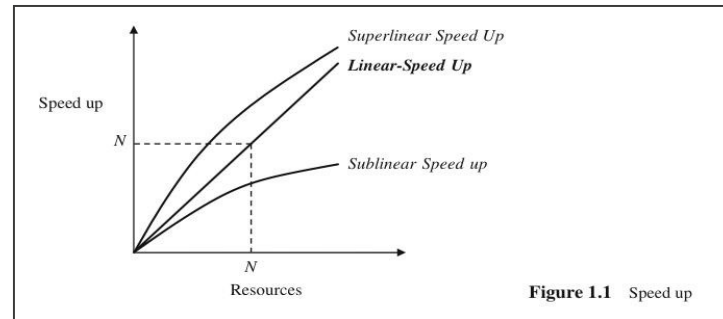
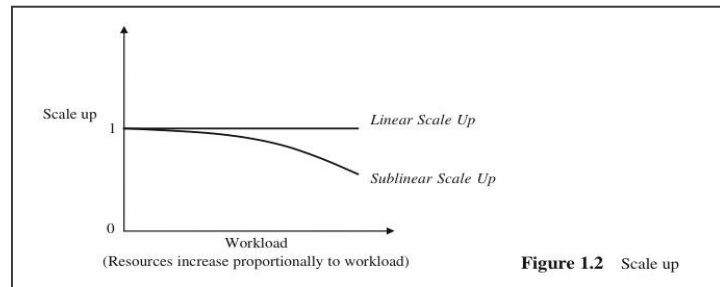


Figure 1.1 Speed up

• Scale up

- Handling of larger tasks by increasing the degree of parallelism
 - The ability to process larger tasks in the same amount of time by providing more resources.
-
- Linear scale up: the ability to maintain the same level of performance when both the workload and the resources are proportionally added
 - Transactional scale up
 - Data scale up

$$\text{Scale up} = \frac{\text{uniprocessor elapsed time on small system}}{\text{multiprocessor elapsed time on larger system}}$$



Quiz 1

Using the current processing resources, we can finish processing 1TB (one terabyte) of data in 1 hour. Recently the volume of data has increased to 2TB and the management has decided to double up the processing resources. Using the new processing resources, we can finish processing the 2 TB in 60 minutes.

Is this **speed up** or **scale up**?

Solution:

$$\text{Scale up} = \frac{\text{uniprocessor elapsed time on small system}}{\text{multiprocessor elapsed time on larger system}}$$

Using x resources (current resources), 1TB = 60 minutes
When the resources are doubled (e.g. x becomes 2x now), a linear scale up is being able to complete 2TB in 60 minutes.

Scale up = 60 mins / 60 mins = 1
i.e. Linear Scale up.

In the question, using 2x resources, it finishes 2 TB in 60 minutes.
Therefore, it is linear scale up.

Parallel Obstacles

• Start-up and Consolidation

- Start up: initiation of multiple processes
- Consolidation: the cost for collecting results obtained from each processor by a host processor

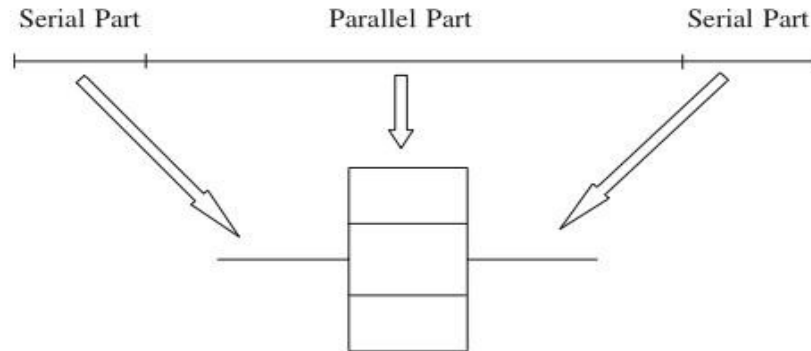


Figure 1.3 Serial part vs. parallel part

Parallel Obstacles

• Interference and Communication

- Interference: competing to access shared resources
- Communication: one process communicating with other processes, and often one has to wait for others to be ready for communication (i.e. waiting time).

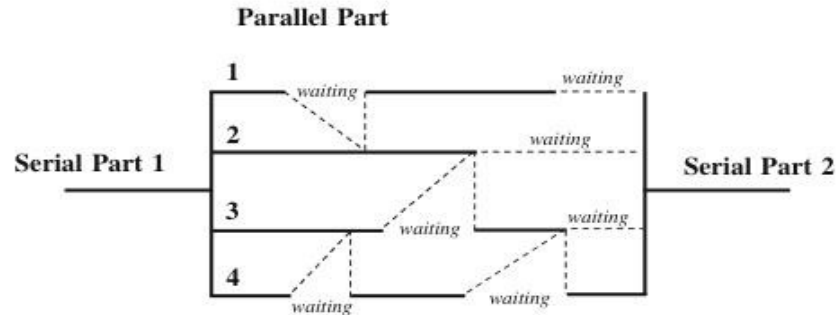


Figure 1.4 Waiting period

Quiz 2

There is a job that will take 1 hour to complete, if this is done by 1 processor. The serial part of this job is 10%. There are 4 processors to use in this job, but each processor will have an overhead of 20% due to **waiting time, communication time, etc.** What type of speed up do we get?

Solution:

1 processor = 60min; Serial part = 10% = 6min; Parallel part = 54min

4 processors = 54min/4 = 13.5min

Overhead = 20%

Hence, parallel processing part = 13.5min + 20% overhead =
13.5min + 2.7min = 16.2min

Total time = 6min + 16.2min = 22.2min

Speed up = 60min / 22.2min = 2.7

Linear speedup should be 4. Speed up of 2.7 is Sub-Linear Speedup

$$\text{Speed up} = \frac{\text{elapsed time on uniprocessor}}{\text{elapsed time on multiprocessors}}$$

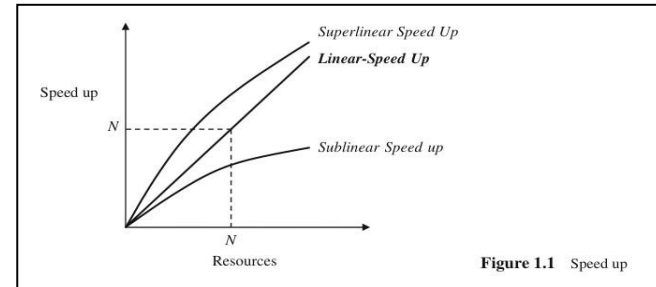


Figure 1.1 Speed up

• Skew

- *Zipf* distribution model to model skew. Measured in terms of different sizes of fragments allocated to the processors

$$|R_i| = \frac{|R|}{i^\theta \times \sum_{j=1}^N \frac{1}{j^\theta}} \quad \text{where } 0 \leq \theta \leq 1 \quad (2.1)$$

- The symbol θ denotes the degree of skewness, where $\theta = 0$ indicates no skew, and **$\theta = 1$ indicates highly skewed**
- $|R|$ is number of records in the table, **$|R_i|$ is number of records in processor i** , and N is number of processor (j is a loop counter, starting from 1 to N)
- Example: $|R|=100,000$ records, $N=8$ processors

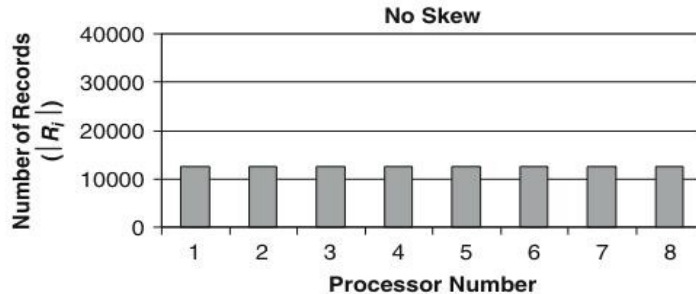


Figure 2.1 Uniform distribution (no skew)

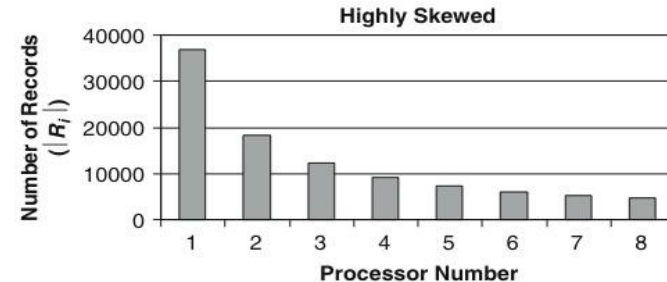


Figure 2.2 Highly skewed distribution

Quiz 3

There 100,000 records in the table to be distributed to 32 processors. Assuming that the **skewness** degree is high ($\theta = 1$), what is the estimated number of records in the heaviest processor?

$$|R_i| = \frac{|R|}{i^\theta \times \sum_{j=1}^N \frac{1}{j^\theta}} \quad \text{where } 0 \leq \theta \leq 1 \quad (2.1)$$

Solution:

$i = 1$ (heaviest processor)

$\theta = 1$

$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{32} = 4.05$

Number of records = $100,000 / 4.05 = 24691$

Skew

■ Data Skew

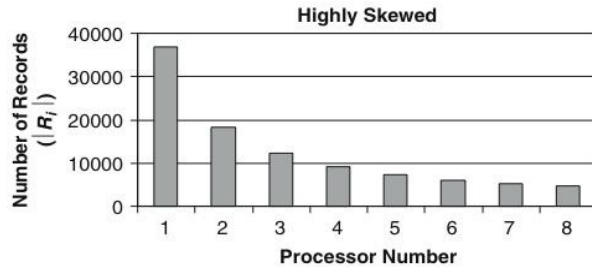


Figure 2.2 Highly skewed distribution

■ Processing Skew

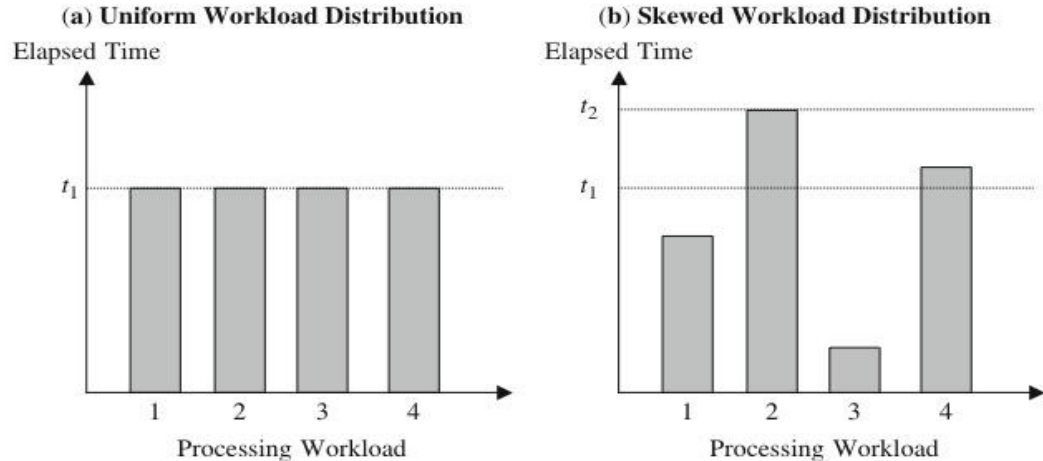
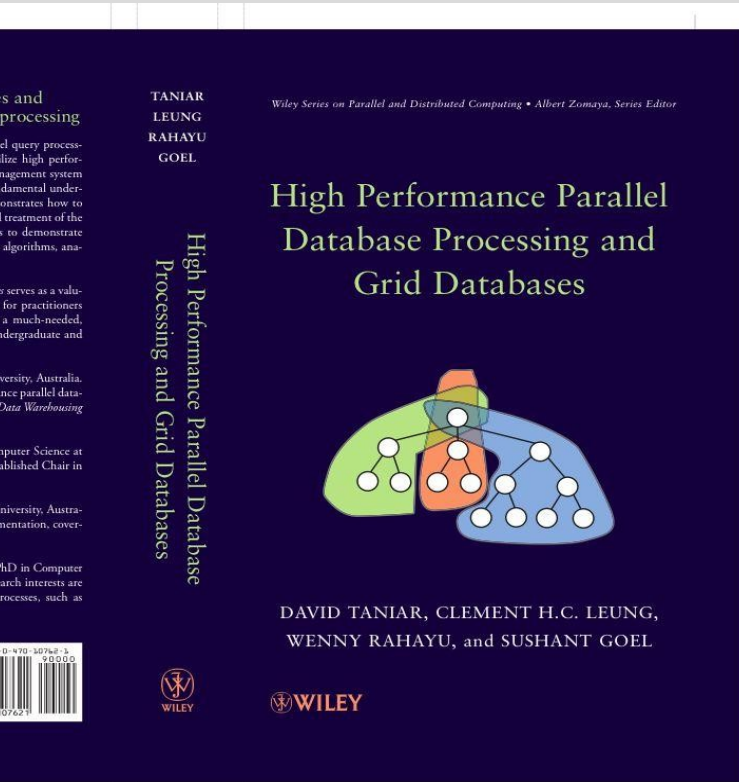


Figure 1.5 Balanced workload vs. unbalanced workload (skewed)

Uneven distribution of data in terms of size.

Uneven processing time of the processors due to data skew.



Chapter 3

Parallel Search

Two steps

- Data partitioning
- Parallel search

- 3.1 Search Queries
- 3.2 Data Partitioning
- 3.3 Search Algorithms
- 3.4 Summary
- 3.5 Bibliographical Notes
- 3.6 Exercises

Data Partitioning

- **Basic Data Partitioning**

- Round-robin or random equal data partitioning
- Hash data partitioning
- Range data partitioning
- Random-unequal data partitioning

- **Complex Data Partitioning**

- Complex data partitioning is based on multiple attributes or is based on a single attribute but with multiple partitioning methods
- Hybrid-Range Partitioning Strategy (HRPS)
- Multiattribute Grid Declustering (MAGIC)

Data Partitioning (cont'd)

Round-robin data partitioning

- Each record in turn is allocated to a processing element in a clockwise manner
- “Equal partitioning” or “Random-equal partitioning”
- Data evenly distributed, hence supports load balance
- But data is not grouped semantically

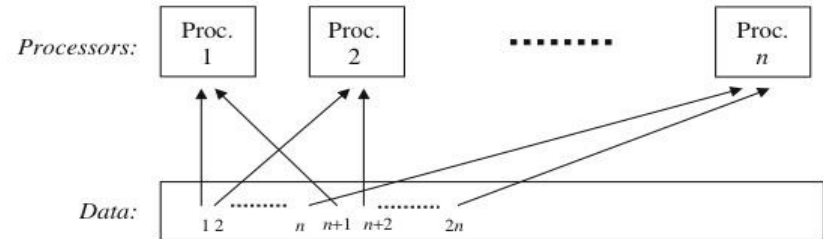


Figure 3.3 Round-robin data partitioning

Data Partitioning (cont'd)

Hash data partitioning

- A hash function is used to partition the data
- Hence, data is grouped semantically, that is data on the same group shared the same hash value
- Selected processors may be identified when processing a search operation (exact-match search), but for range search (especially continuous range), all processors must be used
- Initial data allocation is not balanced either

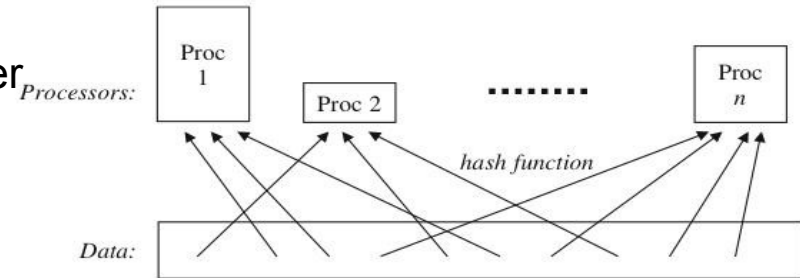


Figure 3.4 Hash data partitioning

The data is divided using a hash function,
can cause skewness

Data Partitioning (cont'd)

Range data partitioning

- Spreads the records based on a given range of the partitioning attribute
- Processing records on a specific range can be directed to certain processor only
- Initial data allocation is skewed too

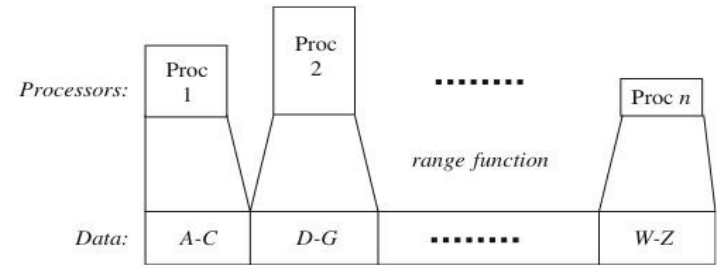


Figure 3.5 Range data partitioning

Search Algorithms

Serial search algorithms:

- Linear search

- Binary search

Parallel search algorithms:

- Processor activation or involvement

- Local searching method

- Key comparison

Search Algorithms (cont'd)

Processor activation or involvement

- The number of processors to be used by the algorithm
- If we know where the data to be sought are stored, then there is no point in activating all other processors in the searching process
- Depends on the data partitioning method used
- Also depends on what type of selection query is performed

Table 3.6 Processor activation or involvement of parallel search algorithms

		Data Partitioning Methods			
		Random-Equal	Hash	Range	Random-Unequal
Exact Match		All	1	1	All
Range Selection	Continuous	All	All	Selected	All
	Discrete	All	Selected	Selected	All

How many processors we need to activate to do the search?

Search Algorithms (cont'd)

Local searching method

- The searching method applied to the processor(s) involved in the searching process
- Depends on the data ordering, regarding the type of the search (exact match of range)

Table 3.7 Local searching method of parallel search algorithms

		Records Ordering	
		Ordered	Unordered
Exact Match		Binary Search	Linear Search
Range Selection	Continuous	Binary Search	Linear Search
	Discrete	Binary Search	Linear Search

Key idea: if the data is sorted, we use binary.
If not we used linear.

Search Algorithms (cont'd)

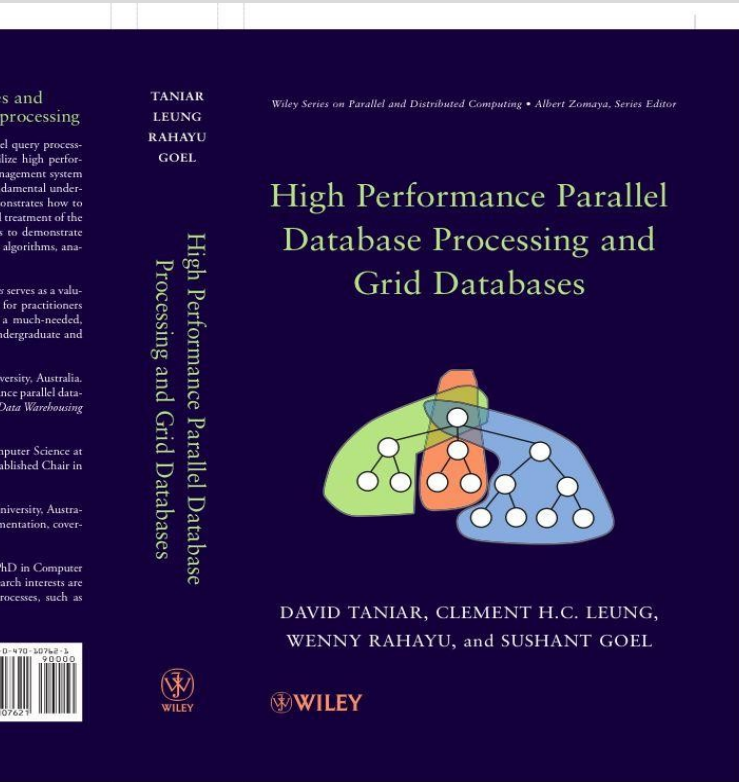
Key comparison

- Compares the data from the table with the condition specified by the query
- When a match is found: continue to find other matches, or terminate
- Depends on whether the data in the table is unique or not

Table 3.8 Key comparison of parallel search algorithms

		Search Attribute Values	
		Unique	Duplicate
Exact Match		Stop	Continue
Range Selection	Continuous	Continue	Continue
	Discrete	Continue	Continue

Should we stop the searching when the match is found?



Chapter 5 Parallel Join

- 5.1 Join Operations
- 5.2 Serial Join Algorithms
- 5.3 Parallel Join Algorithms
- 5.4 Cost Models
- 5.5 Parallel Join Optimization
- 5.6 Summary
- 5.7 Bibliographical Notes
- 5.8 Exercises

Join Algorithms

- Parallel Inner Join components
 - **Data Partitioning**
 - Divide and Broadcast
 - Disjoint Partitioning
 - **Local Join**
 - Nested-Loop Join
 - Sort-Merge Join
 - Hash Join
- Example of a Parallel Inner Join Algorithm
 - **Divide and Broadcast, plus Hash Join**

Serial Join Algorithms (cont'd)

Hash-based Join Algorithm

- The records of files R and S are both hashed to the *same hash file*, using the *same hashing function* on the join attributes A of R and B of S as hash keys
- A single pass through the file with fewer records (say, R) hashes its records to the hash file buckets
- A single pass through the other file (S) then hashes each of its records to the appropriate bucket, where the record is combined with all matching records from R

Serial Join Algorithms (cont'd)

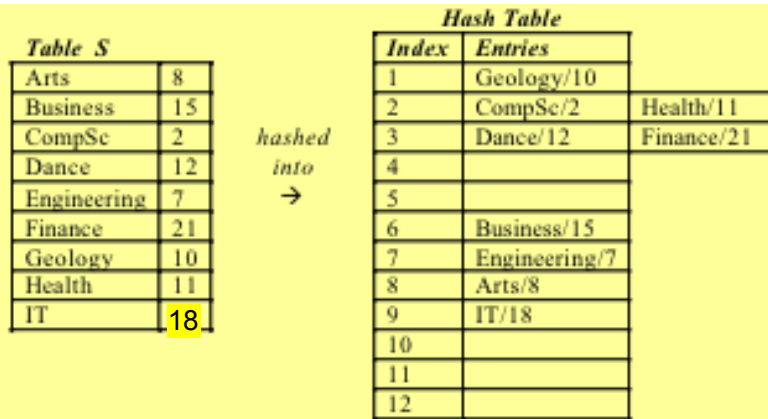


Figure 5.6. Hashing Table S

Serial Join Algorithms (cont'd)

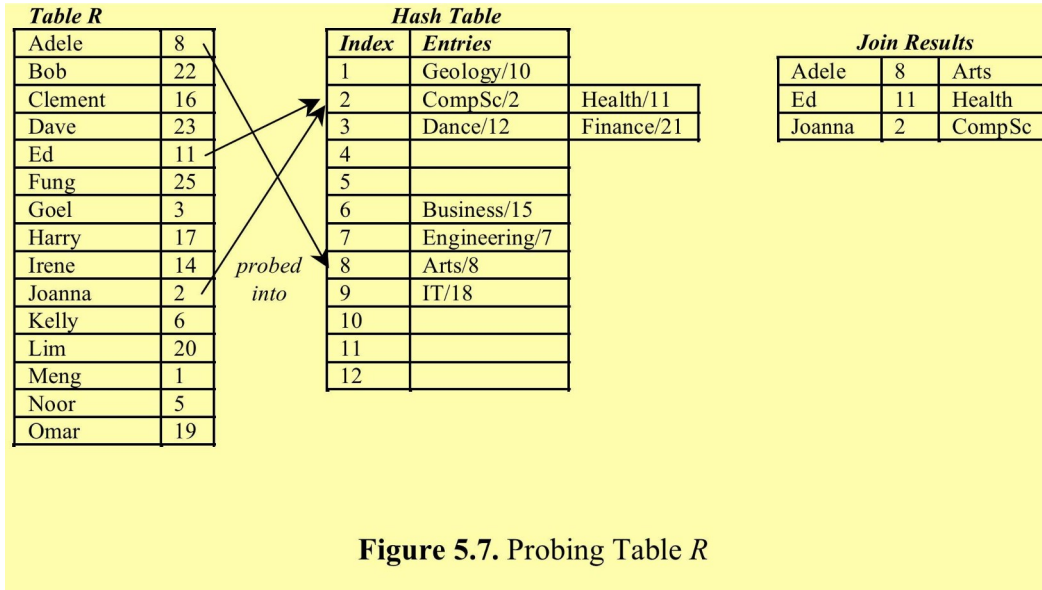


Figure 5.7. Probing Table R

Parallel Join Algorithms (cont'd)

Divide and Broadcast-based Parallel Join Algorithms

- Two stages: data partitioning using the divide and broadcast method, and a local join
- Divide and Broadcast method: **Divide one table into multiple disjoint partitions, where each partition is allocated a processor, and broadcast the other table to all available processors**
- Dividing one table can simply use equal division
- Broadcast means replicate the table to all processors
- Hence, choose the smaller table to broadcast and the larger table to divide

Parallel Join Algorithms (cont'd)

Processor 1				Processor 2				Processor 3			
R1		S1		R2		S2		R3		S3	
Adele	8	Arts	8	Fung	25	Business	12	Kelly	6	CompSc	2
Bob	22	Dance	15	Goel	3	Engineering	7	Lim	20	Finance	21
Clement	16	Geology	10	Harry	17	Health	11	Meng	1	IT	18
Dave	23			Irene	14			Noor	5		
Ed	11			Joanna	2			Omar	19		

Figure 5.10 Initial data placement

Parallel Join Algorithms (cont'd)

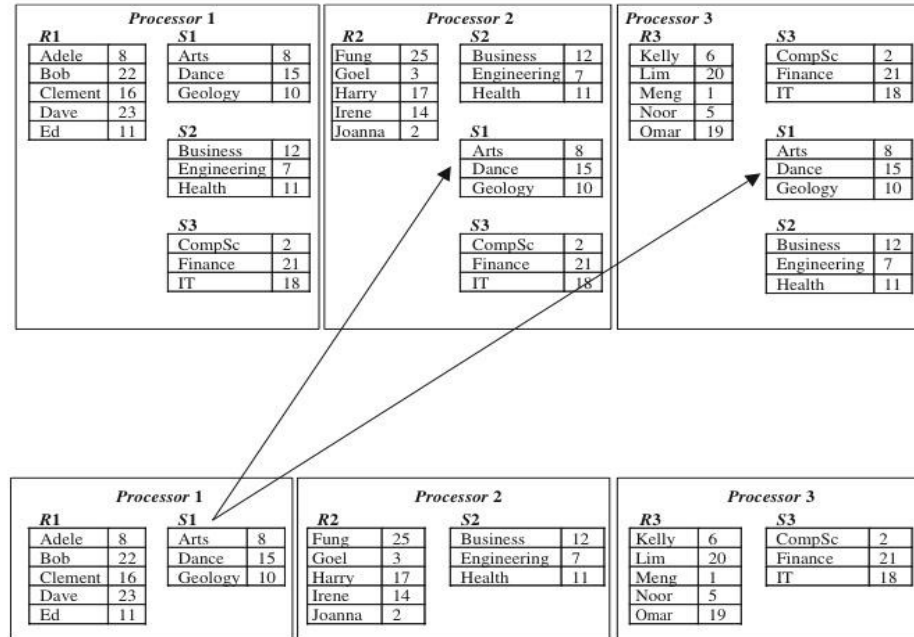


Figure 5.11 Divide and broadcast result

Parallel Join Algorithms (cont'd)

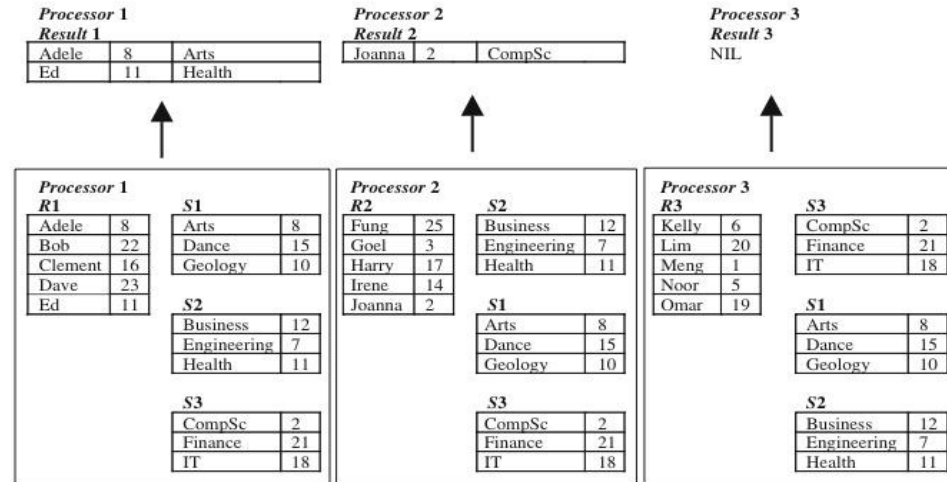


Figure 5.12 Join results based on divide and broadcast

Cost Models?

- **Divide and Broadcast**

- Join two tables (**table R** and **table S**)
- The two tables have been partitioned and stored in 3 processors
- The tables have been partitioned using the random-equal data partitioning method
- The table fragments are called R1, R2, R3, and S1, S2, S3 (in general, each fragment is called **R_i** or **S_i**, where i is the processor number)

Quiz 4

$|S| = 600$ records, each record has the length of 100 bytes, and $N=3$.

Solution:

$S = 60000$ bytes

$S_i = S/N = 60000/3 = 20000$ bytes

$|S| = 600$ records

$|S_i| = 600/3 = 200$ records

Divide and Broadcast based parallel join

Transfer cost = $(S_i/P) \times (N-1) \times (mp+ml)$

Receiving cost = $(S/P - S_i/P) \times (mp)$

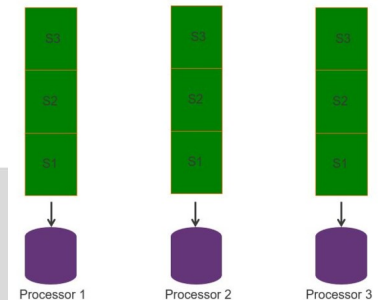
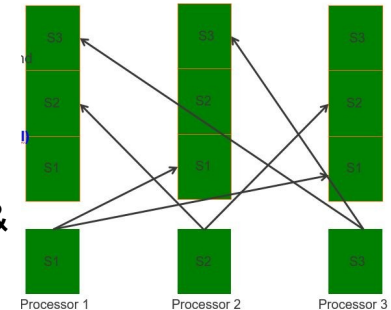
Table 2.1 Cost notations

Symbol	Description
Data parameters	
R	Size of table in bytes
R_i	Size of table fragment in bytes on processor i
$ R $	Number of records in table R
$ R_i $	Number of records in table R on processor i
Systems parameters	
N	Number of processors
P	Page size
H	Hash table size
Query parameters	
π	Projectivity ratio
σ	Selectivity ratio
Time unit cost	
IO	Effective time to read a page from disk
t_r	Time to read a record in the main memory
t_w	Time to write a record to the main memory
t_d	Time to compute destination
Communication cost	
m_p	Message protocol cost per page
m_l	Message latency for one page

Cost Models for Parallel Join

Cost Models for Divide and Broadcast

- Phase 1: data loading
 - *Scan cost* for loading data from local disk in each processor is: $(S_i / P) \times IO$
 - *Select cost* for getting record out of data page is: (read and written by CPU to memory): $|S_i| \times (tr + tw)$
- Phase 2: The broadcast cost by each processor broadcasting its fragment to all other processors
 - *Data transfer cost* is: $(S_i / P) \times (N - 1) \times (mp + ml)$ – mp message protocol & latency
 - *Receiving records cost* is: $(S/P - S_i/P) \times (mp)$
- Phase 3: Each processor after receiving all other fragments of table S , needs to be stored on local disk.
 - *Disk cost for storing the table* is: $(S/P - S_i/P) \times IO$



Query Parameters

- **Projectivity ratio π :**
 - Ratio between projected attribute size and original record length
- **Selectivity ratio σ :**
 - Ratio between number of records in the query result and original total number of records

Example: Join selectivity ratio

If the query operation involves two tables (like in a join operation), a selectivity ratio can be written as σ_j , for example. The value of σ_j indicates the ratio between the number of records produced by a join operation and the number of records of the Cartesian product of the two tables to be joined. For example, $|R_i| = 1000$ records and $|S_i| = 500$ records; if the join produces 5 records only, then the join selectivity ratio σ_i is $5/(1,000 \times 500) = 0.00001$.

Parallel Join Query Processing

Parallel Outer Join processing methods

ROJA (Redistribution Outer Join Algorithm)

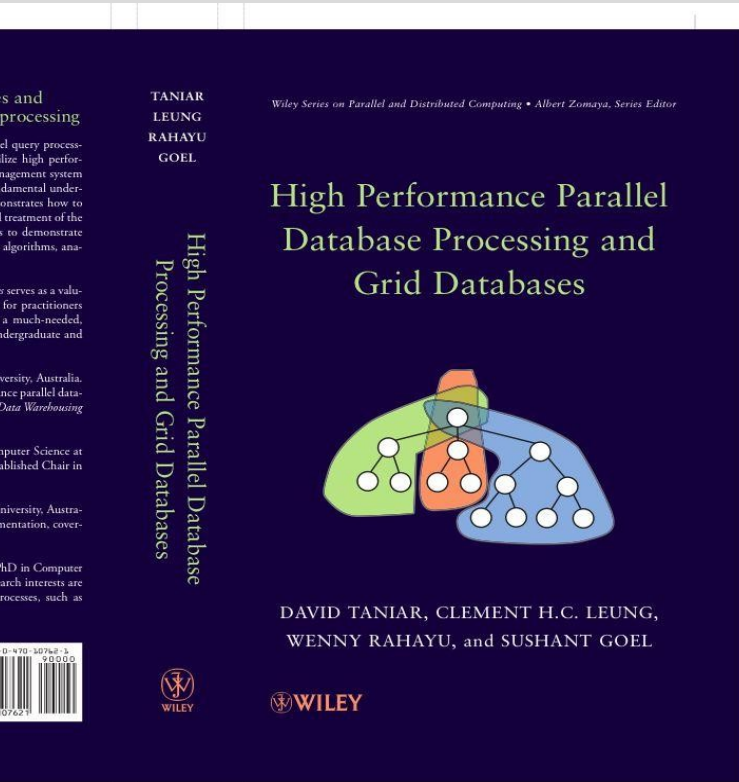
DOJA (Duplication Outer Join Algorithm)

DER (Duplication & Efficient Redistribution)

Load Balancing

OJSO (Outer Join Skew Optimization)

	ROJA	DOJA	DER
Steps	Step 1: Distribute or reshuffle the data based on the join attribute. Step 2: Each processor performs the Local outer Join.	Step 1: Replication. We duplicate the small table. Step 2: Local Inner Join Step 3: Hash redistribute the inner join result based on attribute X. Step 4: Local outer join	Step 1: Replication. We broadcast the left table. Step 2: Local Inner Join Step 3: Select the ROW ID of left table with no matches. Step 4: Redistribute the ROW ID. Step 5: Store the ROW ID that appears as many times as the number of processors. Step 6: Inner join
Pros	fast performance, only two steps	None. ROJA is faster than DOJA.	Redistributes dangling row IDs instead of actual records.
Cons	redistribution of data -> data skew, communication cost	In the replication step, if the table is large, the replication cost is expensive. In the distribution step, data skew and communication cost similar to ROJA	In the replication step, if the table is large, the replication cost is expensive.



Chapter 4

Parallel Sort and GroupBy

- 4.1 Sorting, Duplicate Removal and Aggregate
- 4.2 Serial External Sorting Method
- 4.3 Algorithms for Parallel External Sort
- 4.4 Parallel Algorithms for GroupBy Queries
- 4.5 Cost Models for Parallel Sort
- 4.6 Cost Models for Parallel GroupBy
- 4.7 Summary
- 4.8 Bibliographical Notes

Sorting, and Serial Sorting

- **Serial Sorting – INTERNAL**
 - The data to be sorted fits entirely into the main memory
 - Bubble Sort
 - Insertion Sort
 - Quick Sort
- **Serial Sorting - EXTERNAL**
 - The data to be sorted DOES NOT fit entirely into the main memory
 - Sort-Merge

Quiz 5

There are 150 data pages to be **sorted**. The machine that we have has a limited memory, and can only take 8 pages at a time.

How many passes will it take to sort the 150 data pages?

Solution:

File size to be sorted = 150 pages, number of buffer (or memory size) = 8 pages

Number of subfiles = $150/8 = 19$ subfiles (Last subfile has only 6 pages)

Pass 0 (sorting phase): For each subfile, read from disk, sort in main-memory, and write to disk

Merging phase: We use **7 buffers for input** and **1 buffer for output**

Pass 1: Read 7 sorted subfiles and perform 7-way merging. Repeat the 7-way merging until all subfiles are processed. Result = 3 subfiles

Pass 2: Merge the 3 subfiles

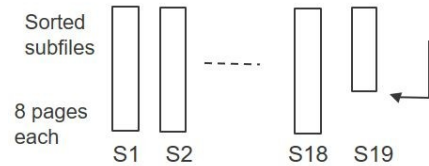
Summary: 150 pages and 8 buffer pages require 3 passes

File size to be sorted = 150 pages, number of buffer (or memory size) = 8 pages

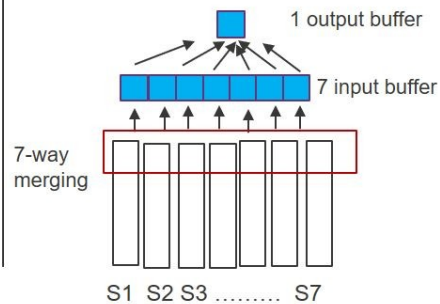
Pass 0 (sorting phase):

Number of subfiles =
 $150/8 = 19$ subfiles
(the last subfile is
only 6 pages long).

Number of
buffer
(memory) =
8 pages



Pass 1 (Merging phase):



S1-S7
S8-S14 } 7-way merging → SS1 SS2
7 X 8 pages =
56 pages each

S15-S18
& S19 } 5-way merging → SS3
4 X 8 + 6 pages
= 38 pages

Pass 2 (Merging phase):

SS1 & SS2 & SS3 $\xrightarrow{\text{3-way merging}}$ SSS1 (sorted array)

Parallel External Sort

- Parallel Merge-All Sort
- Parallel Binary-Merge Sort
- Parallel Redistribution Binary-Merge Sort
- Parallel Redistribution Merge-All Sort
- Parallel Partitioned Sort

Parallel External Sort (cont'd)

Parallel Merge-All Sort

- A traditional approach
- Two phases: local sort and final merge
- Load balanced in local sort
- Problems with merging:
 - Heavy load on one processor
 - Network contention

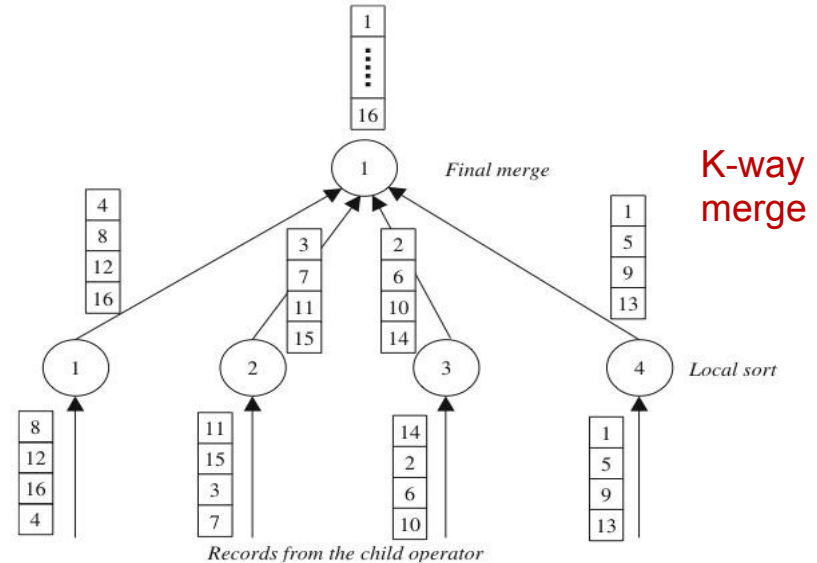


Figure 4.3 Parallel merge-all sort

Parallel External Sort (cont'd)

- Parallel Binary-Merge Sort**

- Local sort similar to traditional method
- Merging in pairs only
- Merging work is now spread to pipeline of processors, but merging is still heavy

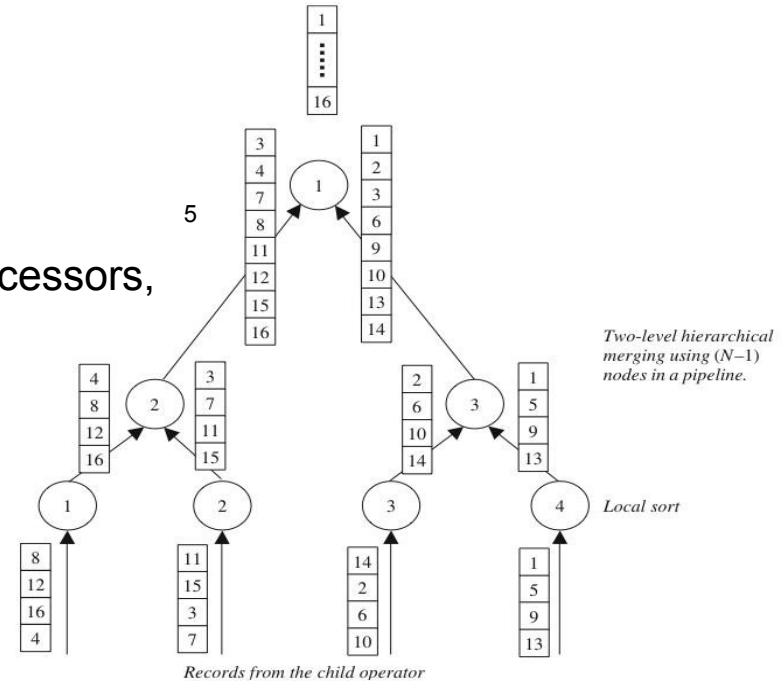


Figure 4.4 Parallel binary-merge sort

Parallel Redistribution Binary-Merge Sort

Parallelism at all levels in the pipeline hierarchy

Step 1: local sort

Step 2: redistribute the results of local sort

Step 3: merge using the same pool of processors

Benefit: merging becomes lighter than without redistribution

Problem: height of the tree

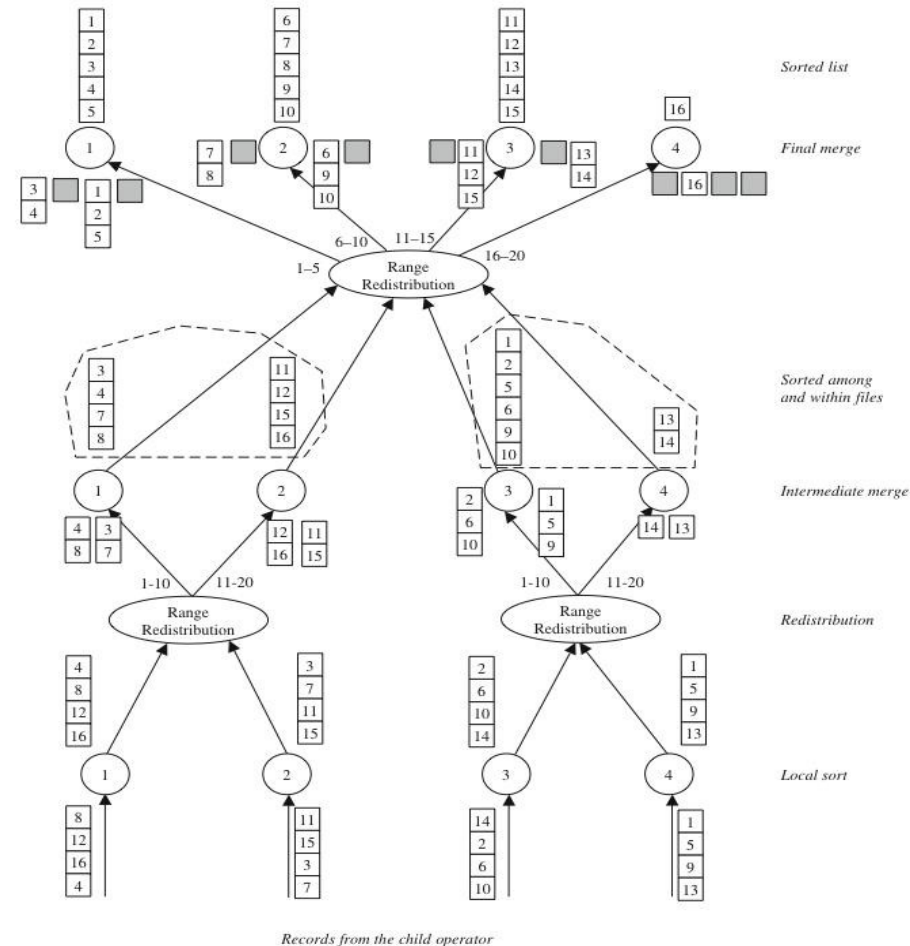


Figure 4.6 Parallel redistribution binary-merge sort

Parallel Redistribution Merge-All Sort

- Reduce the height of the tree, and still maintain parallelism
- Like parallel merge-all sort, but with redistribution
- The advantage is true parallelism in merging
- Skew problem in the merging

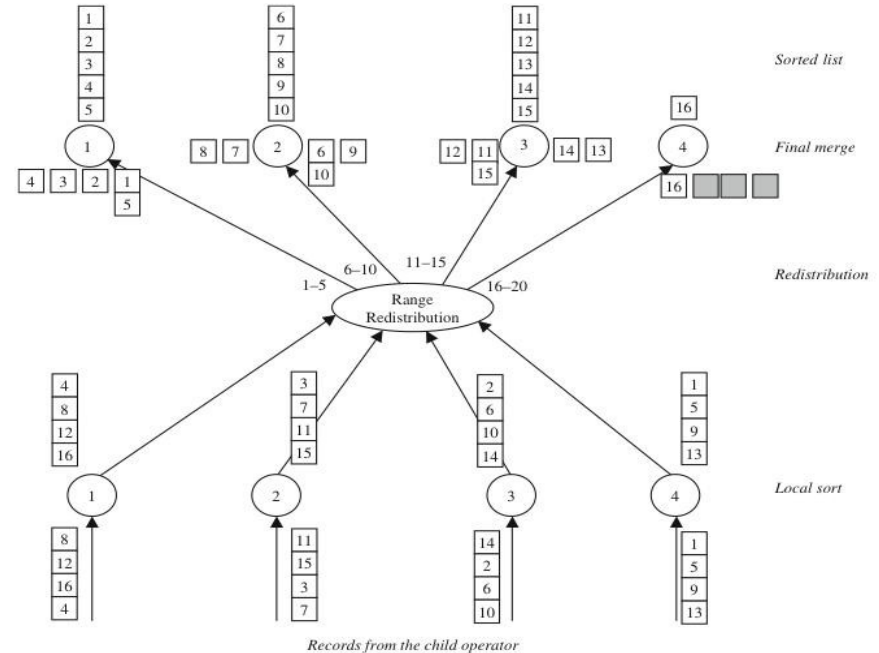


Figure 4.7 Parallel redistribution merge-all sort

Parallel Partitioned Sort

- Two stages: Partitioning stage and Independent local work
- Partitioning (or range redistribution) may raise load skew
- Local sort is done after the partitioning, not before
- No merging is necessary
- Main problem: **Skew** produced by the partitioning

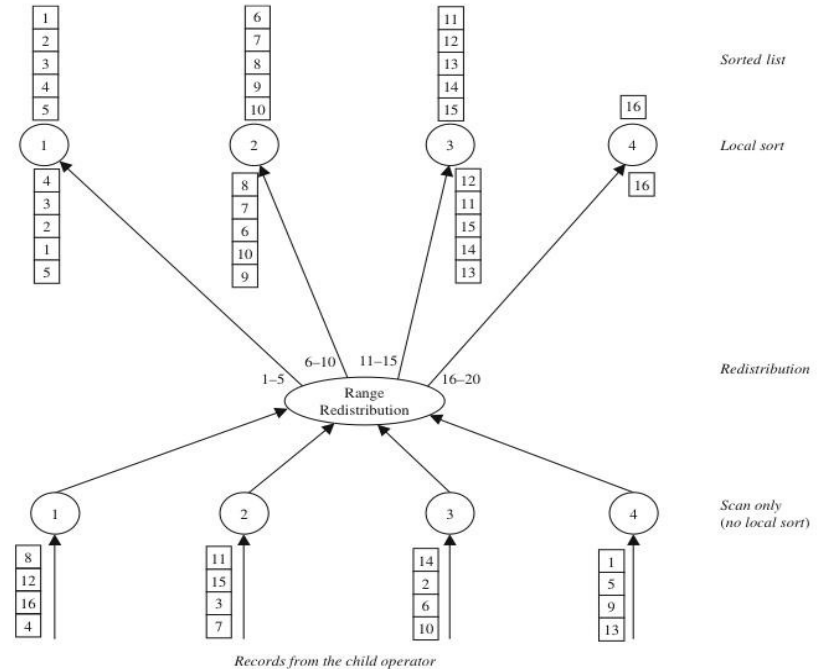


Figure 4.8 Parallel partitioned sort

Parallel External Sort

- **Exercise**

- Given a data set $D = \{8, 11, 14, 1, 12, 15, 2, 5, 16, 3, 6, 9, 4, 7, 10, 13\}$ and four processors, show step by step how the **Parallel Partitioned Sort** works.
- **Initial Data Partitioning** (Round Robin):
 - $P1 = \{8, 12, 16, 4\}$, $P2 = \{11, 15, 3, 7\}$, $P3 = \{14, 2, 6, 10\}$ and $P4 = \{1, 5, 9, 13\}$
- **Range Redistribution** (Range Logic: $P1 = 1-5$, $P2 = 6-10$, $P3 = 11-15$, $P4 = 16-20$)
 - $P1 = \{4, 3, 2, 1, 5\}$, $P2 = \{8, 7, 6, 10, 9\}$, $P3 = \{12, 11, 15, 14, 13\}$, $P4 = \{16\}$
- **Local Sort**
 - $P1 = \{1, 2, 3, 4, 5\}$, $P2 = \{6, 7, 8, 9, 10\}$, $P3 = \{11, 12, 13, 14, 15\}$, $P4 = \{16\}$

Parallel Group By

- Traditional methods (Merge-All and Hierarchical Merging)
- Two-phase method
- Redistribution method

Parallel Group By (cont'd)

Traditional Methods

Step 1: **local aggregate** in each processor

Step 2: **global aggregation**

May use a Merge-All or Hierarchical method

Need to pay a special attention to some aggregate functions (AVG) when performing a local aggregate process

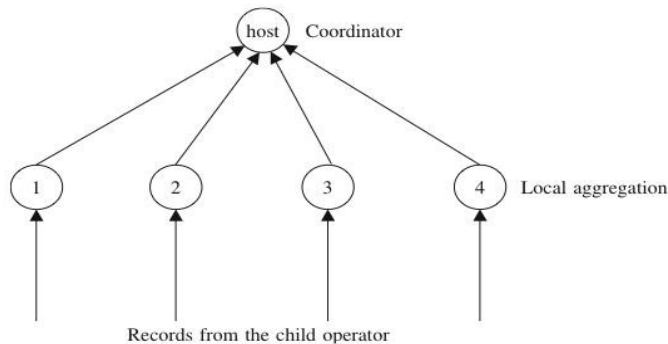


Figure 4.10 Traditional method

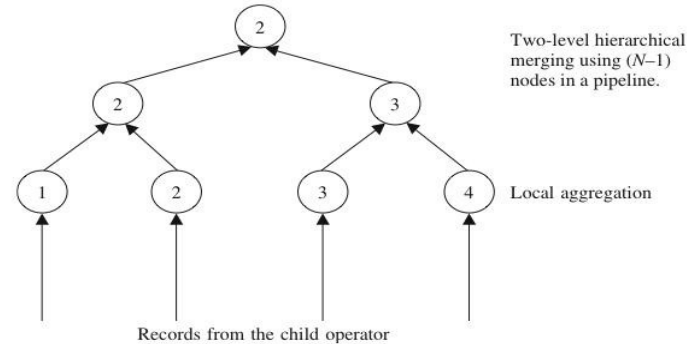


Figure 4.11 Hierarchical merging method

Parallel Group By (cont'd)

Two-Phase Method

Step 1: **local aggregate** in each processor. Each processor groups local records according to the groupby attribute

Step 2: **global aggregation** where all temp results from each processor are redistributed and then final aggregate is performed in each processor

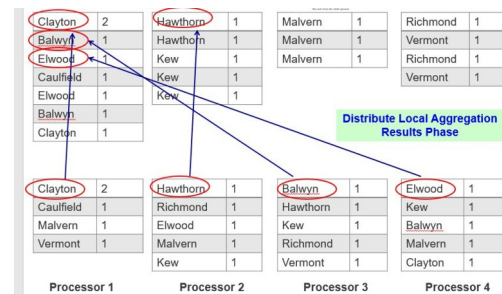
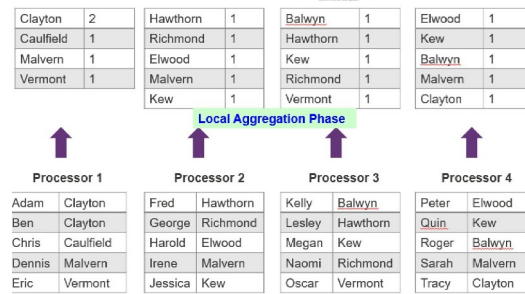
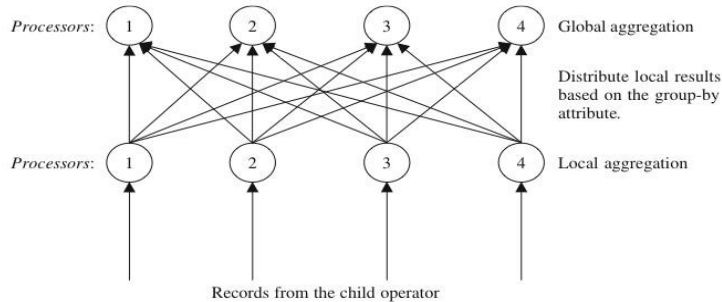
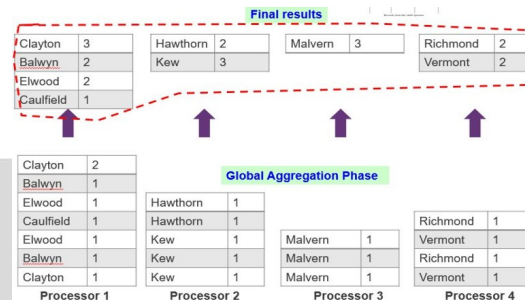


Figure 4.12 Two-phase method



Parallel Group By (cont'd)

Redistribution Method

Step 1 (Partitioning phase): redistribute raw records to all processors

Step 2 (Aggregation phase): each processor performs a local aggregation

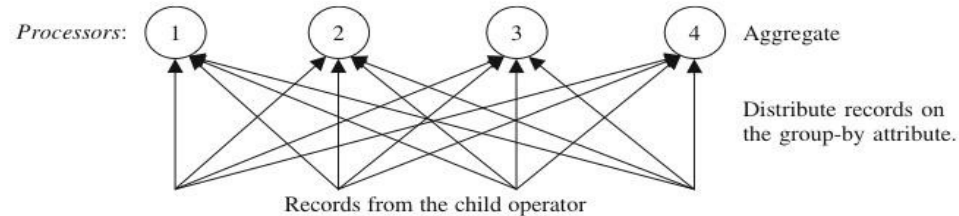


Figure 4.13 Redistribution method

Quiz 7

The **Redistribution Method** has a load balancing option, through the Task Stealing method.

The **Two-Phase Method** does not have a load balancing problem.

Solution: False

Reasons: two phase method also has redistribution step and redistribution of data using range or hash partitioning can easily end up with skew.

Unit Overview

1. **Volume** □ Sessions 1, 2, 3, 4
 - How to process Big Data Volume?
2. **Complexity** □ Sessions 5, 6, 7, 8
 - How to apply machine learning algorithms to every aspect of Big Data?
3. **Velocity** □ Sessions 9, 10, 11
 - How to handle and process Fast Streaming Data?

Machine Learning

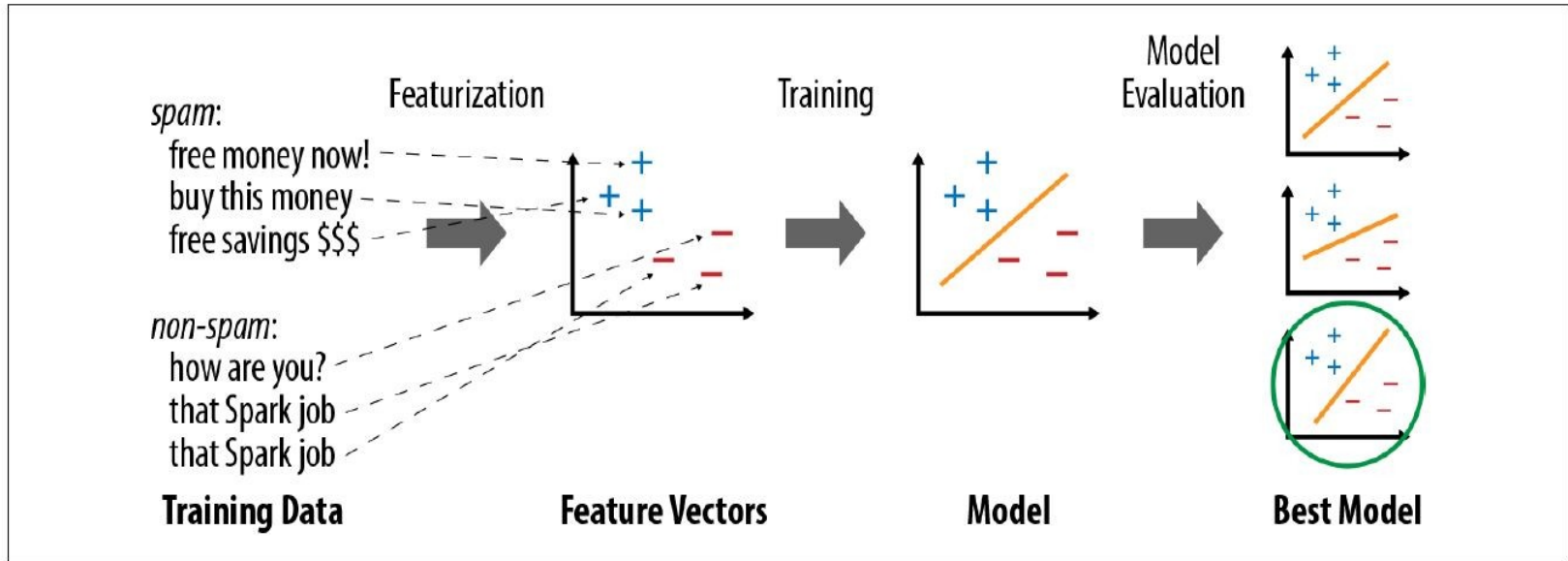


Figure 11-1. Typical steps in a machine learning pipeline

Machine Learning: Featurization

- **Extraction:** Extracting features from “raw” data
 - Count Vectorizer
 - TF-IDF
 - Word2Vec
- **Transformation:** Scaling, converting, or modifying features
 - Tokenization
 - Stop Words Remover
 - String Indexing
 - One Hot Encoding
 - Vector Assembler
- **Selection:** Selecting a subset from a larger set of features
 - Vector Slicer

Quiz 8

In a product recommendation task, simply adding another feature (e.g., realizing that which book you should recommend to a user might also depend on which movies she's watched) could give a large improvement in results.

Solution: True

Types of Machine Learning

- Supervised
- Unsupervised

Types of Machine Learning: **Supervised**

- In supervised machine learning, the data consists of a set of input records.
- Each of these records have **associated labels**.
- The goal is to predict the output label(s) given a new unlabelled input.
- Two types of supervised machine learning:
 1. *Classification* and
 2. *Regression*.

Supervised Machine Learning: **Classification**



Binary classification
example: dog or not
dog

Supervised Machine Learning: **Classification**



Multinomial classification example: Australian shepherd, golden retriever, or poodle

Decision Trees: To Jog or Not To Jog

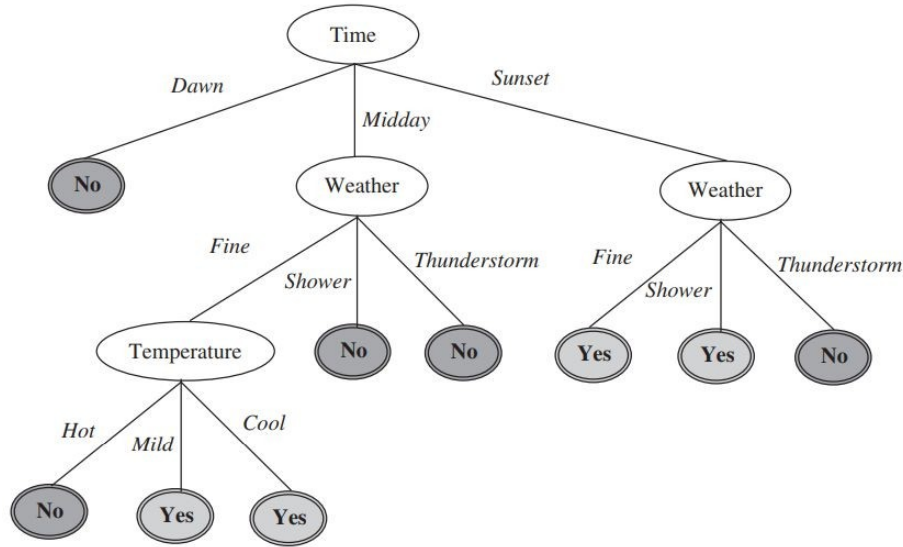


Figure 17.15 Final decision tree

A decision tree is constructed based only on the given training dataset. It is not based on a universal belief.

ID3 (Iterative Dichotomiser 3)

- ❑ It constructs DT, by finding for each node attribute that returns the highest information gain to split the data

■ Steps

1. Compute the entropy for dataset S
2. For every attribute/feature A :
 - 2.1. Calculate entropy for each categorical value of A
 - 2.2 Take weighted average entropy for the current attribute
 - 2.3 Calculate IG for the current attribute
3. Pick the attribute with highest IG to be a node, and split dataset by its branch to child nodes/subsets
4. Repeat same process at every child node until the tree is complete

➡ $H(S)$

➡ $H(S_i)$

➡ $H(S, A) = \sum_{i \in \text{Values}(A)} p_i H(S_i)$

➡ $IG(S, A) = H(S) - H(S, A)$

Stopping condition: when data in each partition have same target class

Quiz 9

What is the **entropy** for the training data set in the table below?

Rec#	Weather	Temperature	Time	Day	Jog (Target Class)
1	Fine	Mild	Sunset	Weekend	Yes
2	Fine	Hot	Sunset	Weekday	Yes
3	Shower	Mild	Midday	Weekday	No
4	Thunderstorm	Cool	Dawn	Weekend	No
5	Shower	Hot	Sunset	Weekday	Yes
6	Fine	Hot	Midday	Weekday	No
7	Fine	Cool	Dawn	Weekend	No
8	Thunderstorm	Cool	Midday	Weekday	No
9	Fine	Cool	Midday	Weekday	Yes
10	Fine	Mild	Midday	Weekday	Yes
11	Shower	Hot	Dawn	Weekend	No
12	Shower	Mild	Dawn	Weekday	No
13	Fine	Cool	Dawn	Weekday	No
14	Thunderstorm	Mild	Sunset	Weekend	No
15	Thunderstorm	Hot	Midday	Weekday	No

Figure 17.11. Training dataset

Entropy: Measure of uncertainty or randomness in data
 Low value -> Less uncertainty, high value -> high uncertainty

$$H(S) = \sum_{i=1}^n p_i \log \frac{1}{p_i} \quad \begin{array}{l} p_i \text{ - Probability of event } i \\ n \text{ - Number of events} \end{array}$$

Information Gain: change in entropy on splitting attribute

$$\begin{aligned}
 IG(S, A) &= H(S) - H(S, A) \\
 &= H(S) - \sum_{i \in \text{Values}(A)} p_i H(S_i)
 \end{aligned}$$

Weighted sum entropy given A

Entropy before (on entire set A)
Entropy after a decision based on A

- Entropy for the given probability of the target classes, $p_1, p_2, \dots, p_n$ where

$\sum_{i=1}^n p_i = 1$, can be calculated as follows:

$$entropy(p_1, p_2, \dots, p_n) = \sum_{i=1}^n (p_i \log(1/p_i)) \quad (17.2)$$

$$\begin{aligned} entropy(Yes, No) &= 5/15 \times \log(15/5) + 10/15 \times \log(15/10) \\ &= 0.2764 \end{aligned} \quad (17.3)$$

- Step 1: Calculate entropy for the training dataset in Figure 17.11. The result is previously calculated as 0.2764 (see equation 17.3).

Rec#	Weather	Temperature	Time	Day	Jog (Target Class)
1	Fine	Mild	Sunset	Weekend	Yes
2	Fine	Hot	Sunset	Weekday	Yes
3	Shower	Mild	Midday	Weekday	No
4	Thunderstorm	Cool	Dawn	Weekend	No
5	Shower	Hot	Sunset	Weekday	Yes
6	Fine	Hot	Midday	Weekday	No
7	Fine	Cool	Dawn	Weekend	No
8	Thunderstorm	Cool	Midday	Weekday	No
9	Fine	Cool	Midday	Weekday	Yes
10	Fine	Mild	Midday	Weekday	Yes
11	Shower	Hot	Dawn	Weekend	No
12	Shower	Mild	Dawn	Weekday	No
13	Fine	Cool	Dawn	Weekday	No
14	Thunderstorm	Mild	Sunset	Weekend	No
15	Thunderstorm	Hot	Midday	Weekday	No

Figure 17.11. Training dataset

Jog	
Yes	No
5	10

$$\begin{aligned} \text{entropy}(\text{Weather}=\text{Fine}) &= 4/7 \times \log(4/7) + 3/7 \times \log(3/7) \\ &= 0.2966 \end{aligned} \quad (17.4)$$

$$\begin{aligned} \text{entropy}(\text{Weather}=\text{Shower}) &= 1/4 \times \log(1/4) + 3/4 \times \log(3/4) \\ &= 0.2442 \end{aligned} \quad (17.5)$$

- Step 2: Process attribute *Weather*
 - Calculate weighted sum entropy of attribute *Weather*:
 $\text{entropy}(\text{Fine}) = 0.2966$
 $\text{entropy}(\text{Shower}) = 0.2442$
 $\text{entropy}(\text{Thunderstorm}) = 0 + 4/4 \times \log(4/4) = 0$
 $\text{weighted sum entropy}(\text{Weather}) = 0.2035$
 - Calculate information gain for attribute *Weather*:
 $\text{gain}(\text{Weather}) = 0.0729$

Rec#	Weather	Temperature	Time	Day	Jog (Target Class)
1	Fine	Mild	Sunset	Weekend	Yes
2	Fine	Hot	Sunset	Weekday	Yes
3	Shower	Mild	Midday	Weekday	No
4	Thunderstorm	Cool	Dawn	Weekend	No
5	Shower	Hot	Sunset	Weekday	Yes
6	Fine	Hot	Midday	Weekday	No
7	Fine	Cool	Dawn	Weekend	No
8	Thunderstorm	Cool	Midday	Weekday	No
9	Fine	Cool	Midday	Weekday	Yes
10	Fine	Mild	Midday	Weekday	Yes
11	Shower	Hot	Dawn	Weekend	No
12	Shower	Mild	Dawn	Weekday	No
13	Fine	Cool	Dawn	Weekday	No
14	Thunderstorm	Mild	Sunset	Weekend	No
15	Thunderstorm	Hot	Midday	Weekday	No

Figure 17.11. Training dataset

		Jog		
		Yes	No	
Weather	Fine	4	3	7
	Shower	1	3	4
	Thunderstorm	0	4	4
				15

$$\begin{aligned}
 \text{Weighted sum entropy (Weather)} &= \text{Weighted entropy (Fine)} \\
 &\quad + \text{Weighted entropy (Shower)} \\
 &\quad + \text{Weighted entropy (Thunderstorm)} \\
 &= 7/15 \times 0.2966 + 4/15 \times 0.2442 + 4/15 \times 0 \\
 &= 0.2035
 \end{aligned}
 \tag{17.6}$$

- Step 2: Process attribute *Weather*
 - Calculate weighted sum entropy of attribute *Weather*:
 $\text{entropy(Fine)} = 0.2966$
 $\text{entropy(Shower)} = 0.2442$
 $\text{entropy(Thunderstorm)} = 0 + 4/4 \times \log(4/4) = 0$
 $\text{weighted sum entropy(Weather)} = 0.2035$
 - Calculate information gain for attribute *Weather*:
 $\text{gain (Weather)} = 0.0729$

Rec#	Weather	Temperature	Time	Day	Jog (Target Class)
1	Fine	Mild	Sunset	Weekend	Yes
2	Fine	Hot	Sunset	Weekday	Yes
3	Shower	Mild	Midday	Weekday	No
4	Thunderstorm	Cool	Dawn	Weekend	No
5	Shower	Hot	Sunset	Weekday	Yes
6	Fine	Hot	Midday	Weekday	No
7	Fine	Cool	Dawn	Weekend	No
8	Thunderstorm	Cool	Midday	Weekday	No
9	Fine	Cool	Midday	Weekday	Yes
10	Fine	Mild	Midday	Weekday	Yes
11	Shower	Hot	Dawn	Weekend	No
12	Shower	Mild	Dawn	Weekday	No
13	Fine	Cool	Dawn	Weekday	No
14	Thunderstorm	Mild	Sunset	Weekend	No
15	Thunderstorm	Hot	Midday	Weekday	No

Figure 17.11. Training dataset

		Jog		
		Yes	No	
Weather	Fine	4	3	7
	Shower	1	3	4
	Thunderstorm	0	4	4
				15

$$\begin{aligned}
 \text{gain}(\text{Weather}) &= \text{entropy}(\text{training dataset } D) - \text{entropy}(\text{attribute } \text{Weather}) \\
 &= 0.2764 - 0.2035 \\
 &= 0.0729
 \end{aligned}
 \tag{17.7}$$

- Step 2: Process attribute *Weather*

- Calculate weighted sum entropy of attribute *Weather*:

$$\text{entropy}(\text{Fine}) = 0.2966$$

(equation 17.4)

$$\text{entropy}(\text{Shower}) = 0.2442$$

(equation 17.5)

$$\text{entropy}(\text{Thunderstorm}) = 0 + 4/4 \times \log(4/4) = 0$$

$$\text{weighted sum entropy}(\text{Weather}) = 0.2035$$

(equation 17.6)

- Calculate information gain for attribute *Weather*:

$$\text{gain}(\text{Weather}) = 0.0729$$

(equation 17.7)

Rec#	Weather	Temperature	Time	Day	Jog (Target Class)
1	Fine	Mild	Sunset	Weekend	Yes
2	Fine	Hot	Sunset	Weekday	Yes
3	Shower	Mild	Midday	Weekday	No
4	Thunderstorm	Cool	Dawn	Weekend	No
5	Shower	Hot	Sunset	Weekday	Yes
6	Fine	Hot	Midday	Weekday	No
7	Fine	Cool	Dawn	Weekend	No
8	Thunderstorm	Cool	Midday	Weekday	No
9	Fine	Cool	Midday	Weekday	Yes
10	Fine	Mild	Midday	Weekday	Yes
11	Shower	Hot	Dawn	Weekend	No
12	Shower	Mild	Dawn	Weekday	No
13	Fine	Cool	Dawn	Weekday	No
14	Thunderstorm	Mild	Sunset	Weekend	No
15	Thunderstorm	Hot	Midday	Weekday	No

Figure 17.11. Training dataset

- Step 3: Process attribute *Temperature*

- Calculate weighted sum entropy of attribute *Temperature*:

$$\text{entropy}(\text{Hot}) = 2/5 \times \log(5/2) + 3/5 \times \log(5/3) = 0.2923$$

$$\text{entropy}(\text{Mild}) = \text{entropy}(\text{Hot})$$

$$\text{entropy}(\text{Cool}) = 1/5 \times \log(5/1) + 4/5 \times \log(5/4) = 0.2173$$

$$\begin{aligned} \text{weighted sum entropy}(\text{Temperature}) &= 5/15 \times 0.2923 + 5/15 \times 0.2173 \\ &= 0.2674 \end{aligned}$$

- Calculate information gain for attribute *Temperature*:

$$\text{gain}(\text{Temperature}) = 0.2764 - 0.2674 = 0.009$$

Rec#	Weather	Temperature	Time	Day	Jog (Target Class)
1	Fine	Mild	Sunset	Weekend	Yes
2	Fine	Hot	Sunset	Weekday	Yes
3	Shower	Mild	Midday	Weekday	No
4	Thunderstorm	Cool	Dawn	Weekend	No
5	Shower	Hot	Sunset	Weekday	Yes
6	Fine	Hot	Midday	Weekday	No
7	Fine	Cool	Dawn	Weekend	No
8	Thunderstorm	Cool	Midday	Weekday	No
9	Fine	Cool	Midday	Weekday	Yes
10	Fine	Mild	Midday	Weekday	Yes
11	Shower	Hot	Dawn	Weekend	No
12	Shower	Mild	Dawn	Weekday	No
13	Fine	Cool	Dawn	Weekday	No
14	Thunderstorm	Mild	Sunset	Weekend	No
15	Thunderstorm	Hot	Midday	Weekday	No

Figure 17.11. Training dataset

		Jog		
		Yes	No	
Temperature	Hot	2	3	5
	Mild	3	2	5
	Cool	1	4	5
				15

- Step 4: Process attribute *Time*

- Calculate weighted sum entropy of attribute *Time*:

$$\text{entropy}(\text{Dawn}) = 0 + 5/5 \times \log(5/5) = 0$$

$$\text{entropy}(\text{Midday}) = 2/6 \times \log(6/2) + 4/6 \times \log(6/4) = 0.2764$$

$$\text{entropy}(\text{Sunset}) = 3/4 \times \log(4/3) + 1/4 \times \log(4/1) = 0.2443$$

$$\text{weighted sum entropy (Time)} = 0 + 6/15 \times 0.2764 + 4/15 \times 0.2443 = 0.1757$$

- Calculate information gain for attribute *Time*:

$$\text{gain (Temperature)} = 0.2764 - 0.1757 = 0.1007$$

Rec#	Weather	Temperature	Time	Day	Jog (Target Class)
1	Fine	Mild	Sunset	Weekend	Yes
2	Fine	Hot	Sunset	Weekday	Yes
3	Shower	Mild	Midday	Weekday	No
4	Thunderstorm	Cool	Dawn	Weekend	No
5	Shower	Hot	Sunset	Weekday	Yes
6	Fine	Hot	Midday	Weekday	No
7	Fine	Cool	Dawn	Weekend	No
8	Thunderstorm	Cool	Midday	Weekday	No
9	Fine	Cool	Midday	Weekday	Yes
10	Fine	Mild	Midday	Weekday	Yes
11	Shower	Hot	Dawn	Weekend	No
12	Shower	Mild	Dawn	Weekday	No
13	Fine	Cool	Dawn	Weekday	No
14	Thunderstorm	Mild	Sunset	Weekend	No
15	Thunderstorm	Hot	Midday	Weekday	No

Figure 17.11. Training dataset

		Jog		
		Yes	No	
Time	Dawn	0	5	5
	Midday	2	4	6
	Sunset	3	1	4
				15

Step 5: Process attribute *Day*

- Calculate weighted sum entropy of attribute *Day*:

$$\text{entropy}(\text{Weekday}) = 4/10 \times \log(10/4) + 6/10 \times \log(10/6) \\ = 0.2923$$

$$\text{entropy}(\text{Weekend}) = 1/5 \times \log(5/1) + 4/5 \times \log(5/4) \\ = 0.2173$$

$$\text{weighted sum entropy}(\text{Day}) = 10/15 \times 0.2923 + 5/15 \\ \times 0.2173 = 0.2674$$

- Calculate information gain for attribute *Day*:

$$\text{gain}(\text{Temperature}) = 0.2764 - 0.2674 = 0.009$$

Rec#	Weather	Temperature	Time	Day	Jog (Target Class)
1	Fine	Mild	Sunset	Weekend	Yes
2	Fine	Hot	Sunset	Weekday	Yes
3	Shower	Mild	Midday	Weekday	No
4	Thunderstorm	Cool	Dawn	Weekend	No
5	Shower	Hot	Sunset	Weekday	Yes
6	Fine	Hot	Midday	Weekday	No
7	Fine	Cool	Dawn	Weekend	No
8	Thunderstorm	Cool	Midday	Weekday	No
9	Fine	Cool	Midday	Weekday	Yes
10	Fine	Mild	Midday	Weekday	Yes
11	Shower	Hot	Dawn	Weekend	No
12	Shower	Mild	Dawn	Weekday	No
13	Fine	Cool	Dawn	Weekday	No
14	Thunderstorm	Mild	Sunset	Weekend	No
15	Thunderstorm	Hot	Midday	Weekday	No

Figure 17.11. Training dataset

		Jog		
		Yes	No	
Day	Weekend	4	6	10
	Weekday	1	4	5
				15

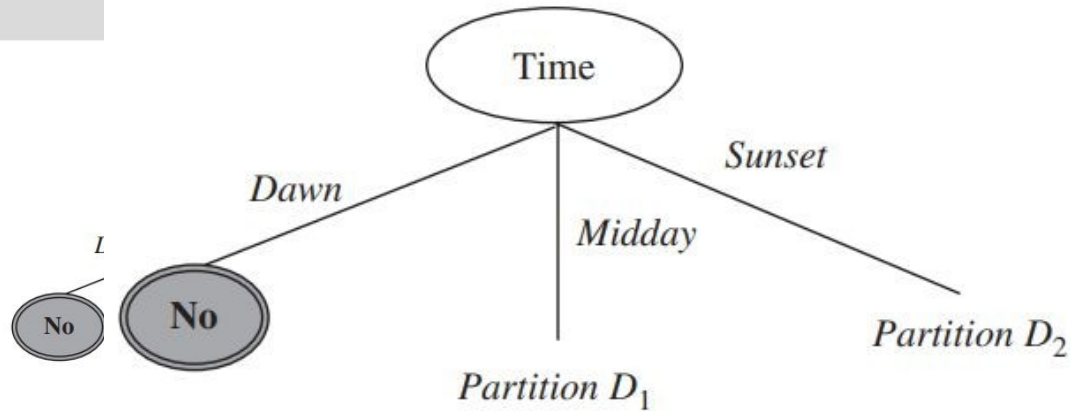


Figure 17.13 Attribute *Time* as the root node

Jog (Target Class)
Yes
Yes
No
No
Yes
No
No
No
Yes
Yes
No
No
No
No

Comparing equations 17.7, 17.8, 17.9, and 17.10, and 17.10 for the gain of each other attributes (Weather, Temperature, Time, and Day), the biggest gain is *Time*, with gain value = 0.1007 (see equation 17.9), and as a result, attribute *Time* is chosen as the first splitting attribute. A partial decision tree with the root node *Time* is shown in Figure 17.13.

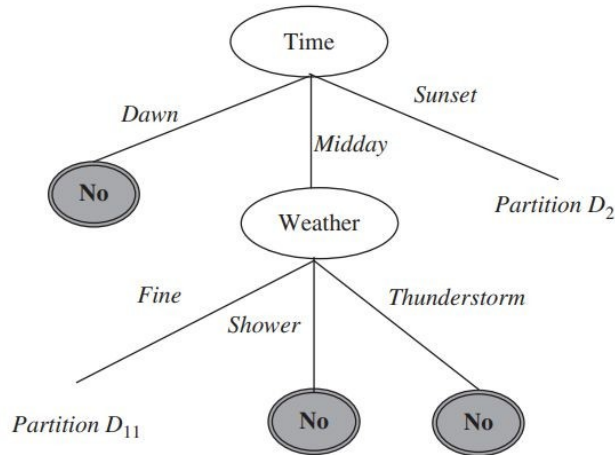


Figure 17.14 Attribute
Weather as next splitting attribute

Rec#	Weather	Temperature	Time	Day	Jog (Target Class)
1	Fine	Mild	Sunset	Weekend	Yes
2	Fine	Hot	Sunset	Weekday	Yes
3	Shower	Mild	Midday	Weekday	No
4	Thunderstorm	Cool	Dawn	Weekend	No
5	Shower	Hot	Sunset	Weekday	Yes
6	Fine	Hot	Midday	Weekday	No
7	Fine	Cool	Dawn	Weekend	No
8	Thunderstorm	Cool	Midday	Weekday	No
9	Fine	Cool	Midday	Weekday	Yes
10	Fine	Mild	Midday	Weekday	Yes
11	Shower	Hot	Dawn	Weekend	No
12	Shower	Mild	Dawn	Weekday	No
13	Fine	Cool	Dawn	Weekday	No
14	Thunderstorm	Mild	Sunset	Weekend	No
15	Thunderstorm	Hot	Midday	Weekday	No

Figure 17.11. Training dataset

consisting of records with
Time=*Midday*. Training dataset partition D_1 consists of 6 records with
record#: 3, 6, 8, 9, 10, and 15. The next task is to determine the splitting
attribute for partition D_1 , whether it is *Weather*, *Temperature*, or *Day*.

Decision Trees: To Jog or Not To Jog

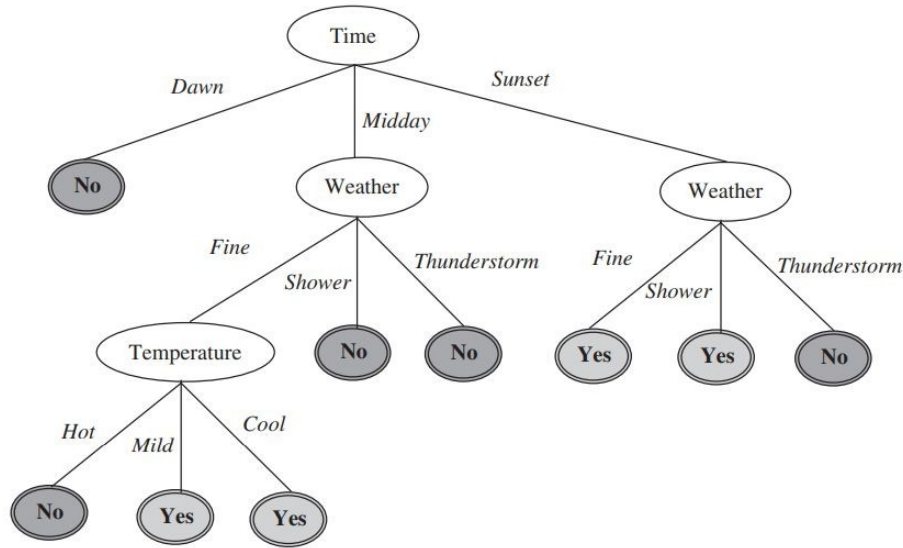
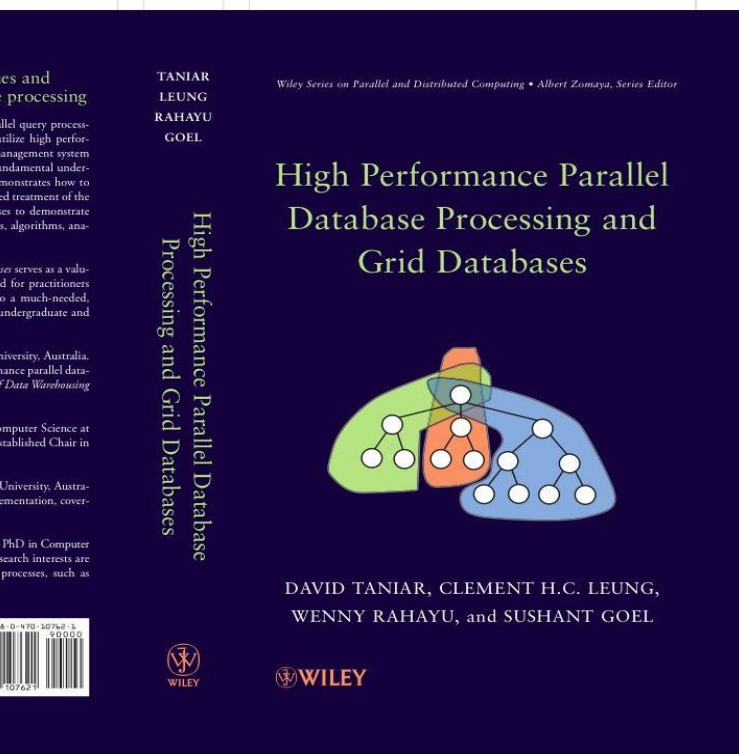


Figure 17.15 Final decision tree

A decision tree is constructed based only on the given training dataset. It is not based on a universal belief.

Chapter 17

Parallel Clustering and Classification



17.1 Clustering and Classification

17.2 Parallel Clustering

17.3 Parallel Classification

17.4 Summary

17.5 Bibliographical Notes

17.6 Exercises

Parallel Classification: Decision Tree

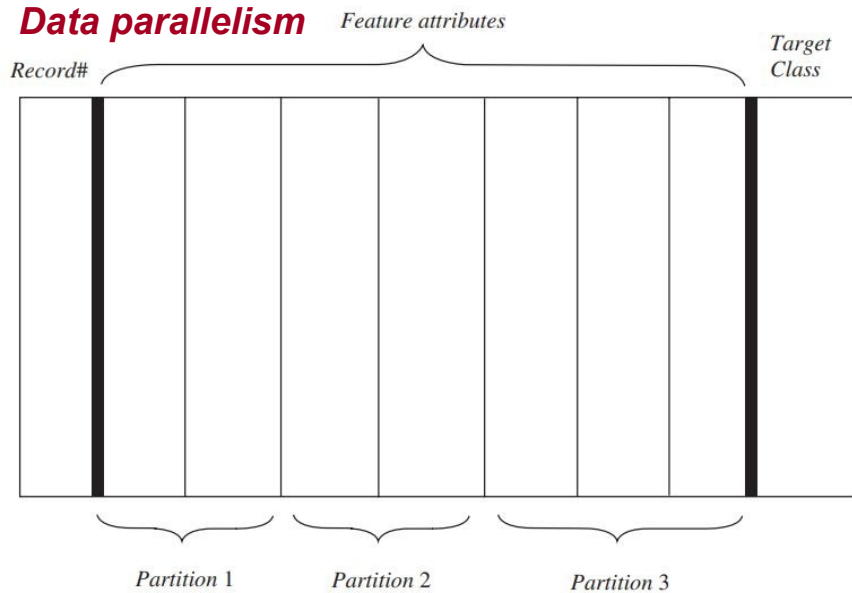


Figure 17.16 Vertical data partitioning of training data set

Rec#	Weather	Temperature	Time	Day	Jog (Target Class)
1	Fine	Mild	Sunset	Weekend	Yes
2	Fine	Hot	Sunset	Weekday	Yes
3	Shower	Mild	Midday	Weekday	No
4	Thunderstorm	Cool	Dawn	Weekend	No
5	Shower	Hot	Sunset	Weekday	Yes
6	Fine	Hot	Midday	Weekday	No
7	Fine	Cool	Dawn	Weekend	No
8	Thunderstorm	Cool	Midday	Weekday	No
9	Fine	Cool	Midday	Weekday	Yes
10	Fine	Mild	Midday	Weekday	Yes
11	Shower	Hot	Dawn	Weekend	No
12	Shower	Mild	Dawn	Weekday	No
13	Fine	Cool	Dawn	Weekday	No
14	Thunderstorm	Mild	Sunset	Weekend	No
15	Thunderstorm	Hot	Midday	Weekday	No

Figure 17.11. Training dataset

Parallel Classification: Decision Tree

Data parallelism:

Rec#	Weather	Temperature	Jog (Target Class)
1	Fine	Mild	Yes
2	Fine	Hot	Yes
3	Shower	Mild	No
4	Thunderstorm	Cool	No
5	Shower	Hot	Yes
6	Fine	Hot	No
7	Fine	Cool	No
8	Thunderstorm	Cool	No
9	Fine	Cool	Yes
10	Fine	Mild	Yes
11	Shower	Hot	No
12	Shower	Mild	No
13	Fine	Cool	No
14	Thunderstorm	Mild	No
15	Thunderstorm	Hot	No

Partition 1

Rec#	Time	Day	Jog (Target Class)
1	Sunset	Weekend	Yes
2	Sunset	Weekday	Yes
3	Midday	Weekday	No
4	Dawn	Weekend	No
5	Sunset	Weekday	Yes
6	Midday	Weekday	No
7	Dawn	Weekend	No
8	Midday	Weekday	No
9	Midday	Weekday	Yes
10	Midday	Weekday	Yes
11	Dawn	Weekend	No
12	Dawn	Weekday	No
13	Dawn	Weekday	No
14	Sunset	Weekend	No
15	Midday	Weekday	No

Partition 2

Parallel Classification: Decision Tree

Data parallelism

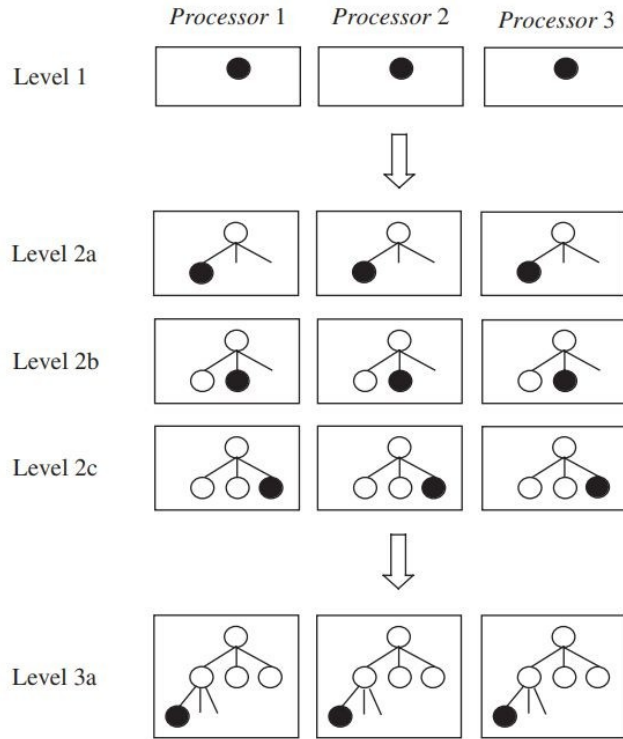


Figure 17.17 Data parallelism of parallel decision tree construction

Rec#	Weather	Temperature	Time	Day	Jog (Target Class)
1	Fine	Mild	Sunset	Weekend	Yes
2	Fine	Hot	Sunset	Weekday	Yes
3	Shower	Mild	Midday	Weekday	No
4	Thunderstorm	Cool	Dawn	Weekend	No
5	Shower	Hot	Sunset	Weekday	Yes
6	Fine	Hot	Midday	Weekday	No
7	Fine	Cool	Dawn	Weekend	No
8	Thunderstorm	Cool	Midday	Weekday	No
9	Fine	Cool	Midday	Weekday	Yes
10	Fine	Mild	Midday	Weekday	Yes
11	Shower	Hot	Dawn	Weekend	No
12	Shower	Mild	Dawn	Weekday	No
13	Fine	Cool	Dawn	Weekday	No
14	Thunderstorm	Mild	Sunset	Weekend	No
15	Thunderstorm	Hot	Midday	Weekday	No

Figure 17.11. Training dataset

- ❑ All processors will **process one particular node at each level or sub-level at a time**
- ❑ Each processor will compute IGs for certain attributes (in parallel), which are then shared to determine splitting
- ❑ 'Intra tree node parallelism'

Parallel Classification: Decision Tree

Result parallelism

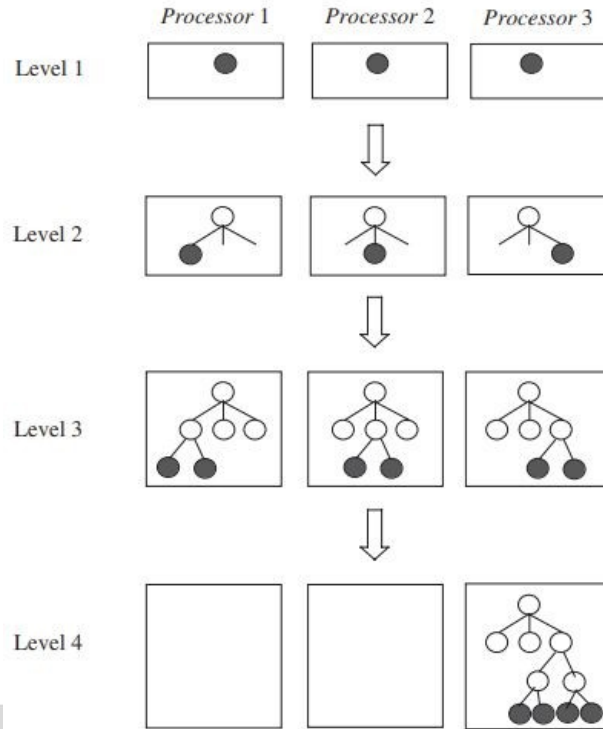


Figure 17.20 Result parallelism of parallel decision tree construction

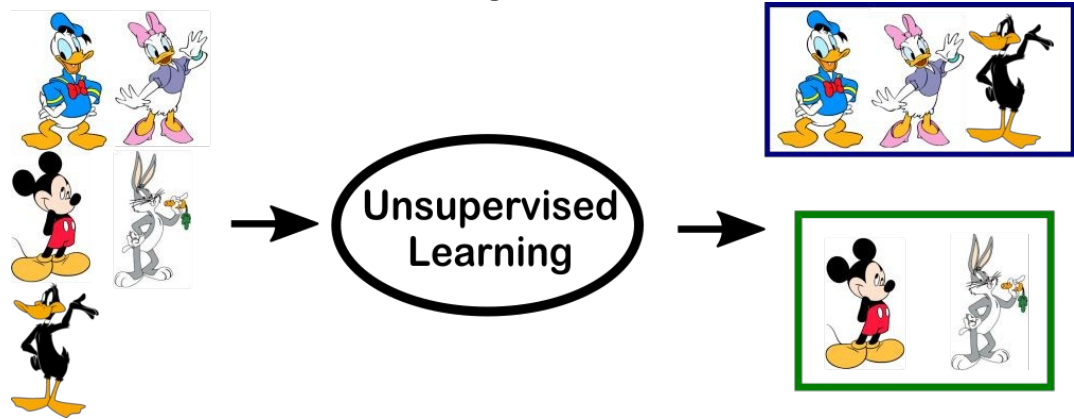
Rec#	Weather	Temperature	Time	Day	Jog (Target Class)
1	Fine	Mild	Sunset	Weekend	Yes
2	Fine	Hot	Sunset	Weekday	Yes
3	Shower	Mild	Midday	Weekday	No
4	Thunderstorm	Cool	Dawn	Weekend	No
5	Shower	Hot	Sunset	Weekday	Yes
6	Fine	Hot	Midday	Weekday	No
7	Fine	Cool	Dawn	Weekend	No
8	Thunderstorm	Cool	Midday	Weekday	No
9	Fine	Cool	Midday	Weekday	Yes
10	Fine	Mild	Midday	Weekday	Yes
11	Shower	Hot	Dawn	Weekend	No
12	Shower	Mild	Dawn	Weekday	No
13	Fine	Cool	Dawn	Weekday	No
14	Thunderstorm	Mild	Sunset	Weekend	No
15	Thunderstorm	Hot	Midday	Weekday	No

Figure 17.11. Training dataset

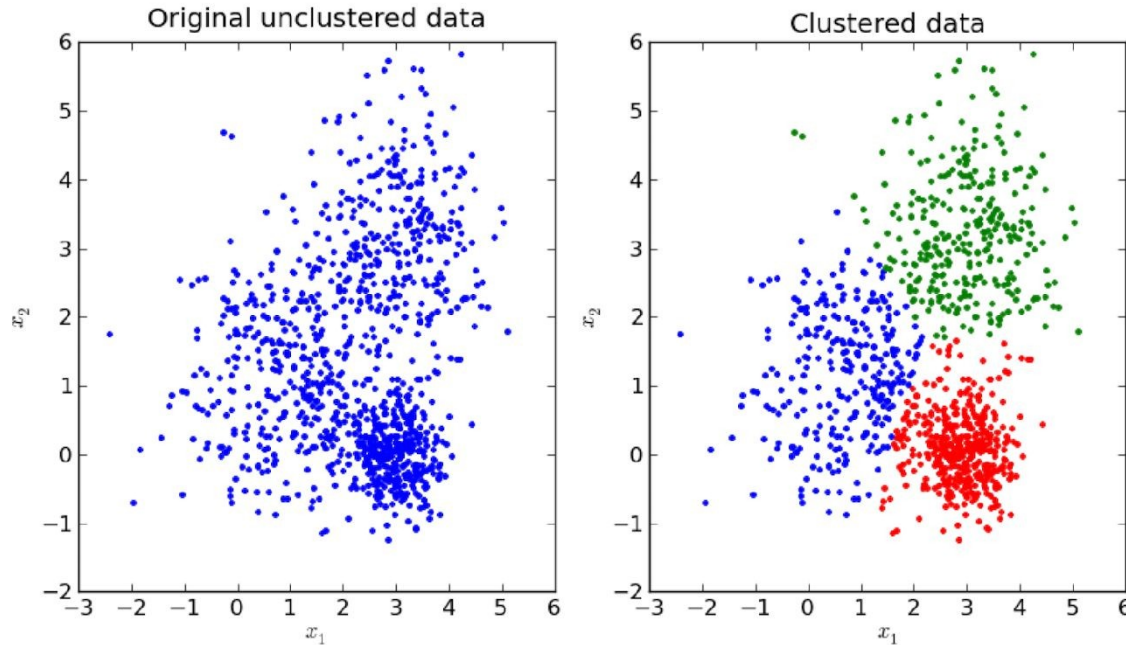
- ❑ Multiple nodes are processed concurrently using several processors at each level
- ❑ Main rule: A processor that processes a child node will also its parent nodes
- ❑ 'Inter tree node parallelism'

Types of Machine Learning: **Unsupervised**

- Instead of predicting a label, unsupervised ML helps you to better understand the structure of your data.
- Two types of unsupervised machine learning:
 1. *Clustering* and
 2. *Association*.



Unsupervised Machine Learning: **Clustering**



Clustering example

• **Algorithm** k-Means:

- (**Initialisation**) Specifies k number of clusters, and guesses the k seed cluster centroid
- (**Assignment Step**) Assign each data point to the cluster with the closest centroid
 - Current clusters may receive or loose their members
- (**Update Step**) Each cluster must re-calculate the mean (centroid)
- The process is repeated until the clusters are stable (no change of members)

Algorithm: k-means

Input:

$D=\{x_1, x_2, \dots, x_n\}$

//Data objects

k

//Number of desired clusters

Output:

K

//Set of clusters

1. Assign initial values for means $m_1, m_2, \dots, m_k$
2. Repeat
3. Assign each data object x_i to the cluster which has the closest mean
4. Calculate new mean for each cluster
5. Until convergence criteria is met

Quiz 10

Data D = {5, 19, 25, 21, 4, 1, 17, 23, 8, 7, 6, 10, 2, 20, 14, 11, 27, 9, 3, 16}

Number of clusters: $k = 3$

Initial centroids: $m_1=6$, $m_2=7$, and $m_3=8$.

Which of the following grouping is correct after applying K-Means algorithm?

Solution:

$$C_1 = \{1, 2, 3, 4, 5, 6\}$$

$$C_2 = \{7, 8, 9, 10, 11, 14\}$$

$$C_3 = \{16, 17, 19, 20, 21, 23, 25, 27\}$$

k-Means: Step-By-Step Example

- Data $D = \{5, 19, 25, 21, 4, 1, 17, 23, 8, 7, 6, 10, 2, 20, 14, 11, 27, 9, 3, 16\}$
- Number of clusters: $k = 3$
- Initial centroids: $m_1=6$, $m_2=7$, and $m_3=8$
- **First Iteration**
 - Clusters:
 - $C_1=\{1, 2, 3, 4, 5, 6\}$
 - $C_2=\{7\}$
 - $C_3=\{8, 9, 10, 11, 14, 16, 17, 19, 20, 21, 23, 25, 27\}$
 - Re-calculated centroids: $m_1=3.5$, $m_2=7$, and $m_3=16.9$

First Iteration: Calculating euclidean distance, determining the cluster membership and calculating new centroid.

D	5	19	25	21	4	1	17	23	8	7	6	10	2	20	14	11	27	9	3	16
$d(m_1, D_i)$	1	13	19	15	2	5	11	17	2	1	0	4	4	14	8	5	21	3	3	10
$d(m_2, D_i)$	2	12	18	14	3	6	10	16	1	0	1	3	5	13	7	4	20	2	4	9
$d(m_3, D_i)$	3	11	17	13	4	7	9	15	0	1	2	2	6	12	6	3	19	1	5	8



k-Means: Step-By-Step Example

- Clusters:
 - $C_1 = \{1, 2, 3, 4, 5, 6\}$
 - $C_2 = \{7\}$
 - $C_3 = \{8, 9, 10, 11, 14, 16, 17, 19, 20, 21, 23, 25, 27\}$
- New centroids: $m_1 = 3.5$, $m_2 = 7$, and $m_3 = 16.9$
- **Second Iteration**
 - Clusters:
 - $C_1 = \{1, 2, 3, 4, 5\}$
 - $C_2 = \{6, 7, 8, 9, 10, 11\}$
 - $C_3 = \{14, 16, 17, 19, 20, 21, 23, 25, 27\}$
 - Re-calculated centroids: $m_1 = 3$, $m_2 = 8.5$, and $m_3 = 20.2$

Second Iteration: Calculating euclidean distance, determining the cluster membership and calculating new centroid.

	5	19	25	21	4	1	17	23	8	7	6	10	2	20	14	11	27	9	3	16
$d(m_1, D_i)$	1.5	15.5	21.5	17.5	0.5	2.5	13.5	19.5	4.5	3.5	2.5	6.5	1.5	16.5	10.5	7.5	23.5	5.5	0.5	12.5
$d(m_2, D_i)$	2	12	18	14	3	6	10	16	1	0	1	3	5	13	7	4	20	2	4	9
$d(m_3, D_i)$	11.9	2.1	8.1	4.1	12.9	15.9	0.1	6.1	8.9	9.9	10.9	6.9	14.9	3.1	2.9	5.9	10.1	7.9	13.9	0.9

k-Means: Step-By-Step Example

- Clusters:
 - $C_1 = \{1, 2, 3, 4, 5\}$
 - $C_2 = \{6, 7, 8, 9, 10, 11\}$
 - $C_3 = \{14, 16, 17, 19, 20, 21, 23, 25, 27\}$
- New centroids: $m_1=3$, $m_2=8.5$, and $m_3=20.2$
- **Third Iteration**
 - Clusters:
 - $C_1 = \{1, 2, 3, 4, 5\}$
 - $C_2 = \{6, 7, 8, 9, 10, 11, 14\}$
 - $C_3 = \{16, 17, 19, 20, 21, 23, 25, 27\}$
 - Re-calculated centroids: $m_1=3$, $m_2=9.29$, and $m_3=21$

Third Iteration: Calculating euclidean distance, determining the cluster membership and calculating new centroid.

	5	19	25	21	4	1	17	23	8	7	6	10	2	20	14	11	27	9	3	16
$d(m_1, D_i)$	2	16	22	18	1	2	14	20	5	4	3	7	1	17	11	8	24	6	0	13
$d(m_2, D_i)$	10.5	16.5	12.5	4.5	7.5	8.5	14.5	0.5	1.5	2.5	1.5	6.5	11.5	5.5	2.5	18.5	0.5	5.5	7.5	
$d(m_3, D_i)$	15.2	1.2	4.8	0.8	16.2	19.2	3.2	2.8	12.2	13.2	14.2	10.2	18.2	0.2	6.2	9.2	6.8	11.2	17.2	4.2

k-Means: Step-By-Step Example

- Clusters:
 - $C_1 = \{1, 2, 3, 4, 5\}$
 - $C_2 = \{6, 7, 8, 9, 10, 11, 14\}$
 - $C_3 = \{16, 17, 19, 20, 21, 23, 25, 27\}$
- New centroids: $m_1 = 3$, $m_2 = 9.29$, and $m_3 = 21$
- **Fourth Iteration**
 - Clusters:
 - $C_1 = \{1, 2, 3, 4, 5, 6\}$
 - $C_2 = \{7, 8, 9, 10, 11, 14\}$
 - $C_3 = \{16, 17, 19, 20, 21, 23, 25, 27\}$
 - Re-calculated centroids: $m_1 = 3.5$, $m_2 = 9.83$, and $m_3 = 21$

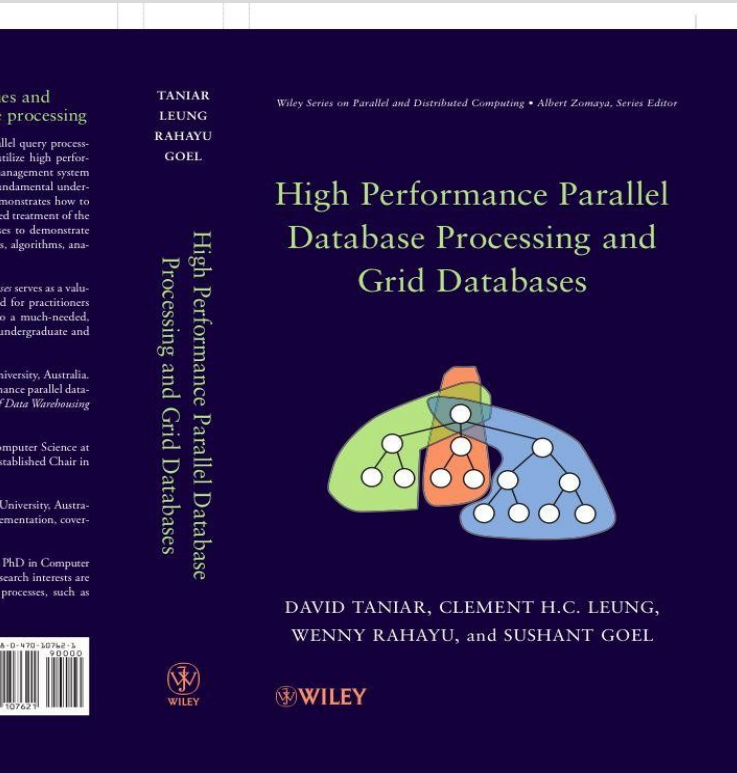
Fourth Iteration: Calculating euclidean distance, determining the cluster membership and calculating new centroid.

	5	19	25	21	4	1	17	23	8	7	6	10	2	20	14	11	27	9	3	16
$d(m_1, D_i)$	2	16	22	18	1	2	14	20	5	24	3	7	1	17	11	8	24	6	0	13
$d(m_2, D_i)$	9.7	15.7	11.7	5.3	8.3	7.7	13.7	1.3	2.3	3.3	0.7	7.3	10.7	4.7	1.7	17.7	0.3	6.3	6.7	
$d(m_3, D_i)$	16.0	2.0	4.0	0.0	17.0	20.0	4.0	2.0	13.0	14.0	15.0	11.0	19.0	1.0	7.0	10.0	6.0	12.0	18.0	5.0

k-Means: Step-By-Step Example

- Clusters:
 - $C_1 = \{1, 2, 3, 4, 5, 6\}$
 - $C_2 = \{7, 8, 9, 10, 11, 14\}$
 - $C_3 = \{16, 17, 19, 20, 21, 23, 25, 27\}$
- New centroids: $m_1 = 3.5$, $m_2 = 9.83$, and $m_3 = 21$
- **Fifth Iteration**
 - No data movement from clusters (Process Terminated)

m_1	m_2	m_3	C_1	C_2	C_3
6	7	8	1, 2, 3, 4, 5, 6	7	8, 9, 10, 11, 14, 16, 17, 19, 20, 23, 25, 27
3.5	7	16.9	1, 2, 3, 4, 5	6, 7, 8, 9, 10, 11	14, 16, 17, 19, 20, 21, 23, 25, 27
3	8.5	20.2	1, 2, 3, 4, 5	6, 7, 8, 9, 10, 11, 14	16, 17, 19, 20, 21, 23, 25, 27
3	9.29	21	1, 2, 3, 4, 5, 6	7, 8, 9, 10, 11, 14	16, 17, 19, 20, 21, 23, 25, 27
3.5	9.83	21	1, 2, 3, 4, 5, 6	7, 8, 9, 10, 11, 14	16, 17, 19, 20, 21, 23, 25, 27



Chapter 17

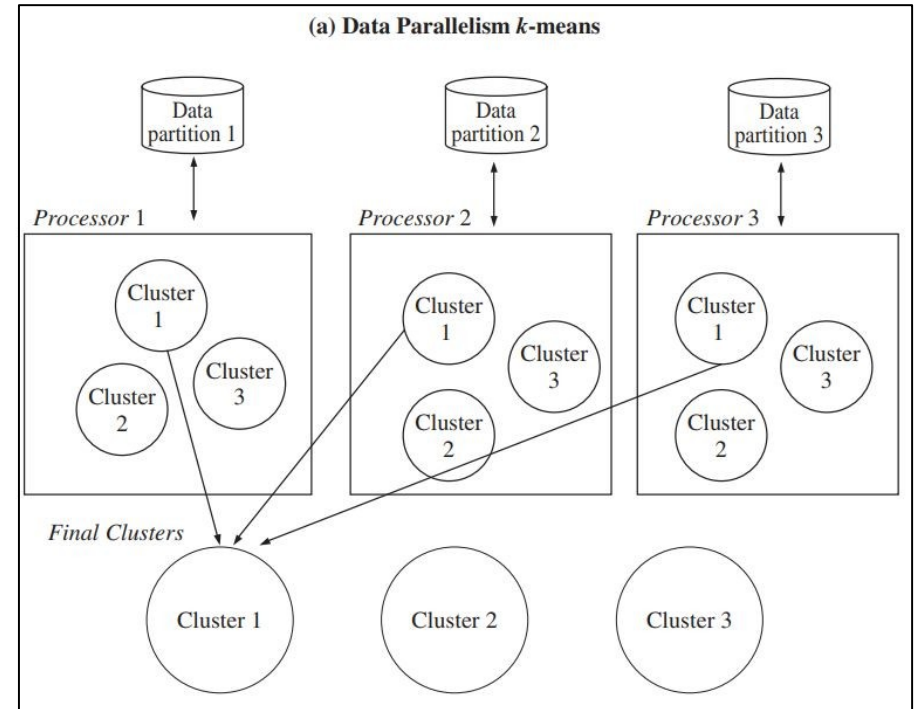
Parallel Clustering and Classification

- 17.1 Clustering and Classification
- 17.2 Parallel Clustering
- 17.3 Parallel Classification
- 17.4 Summary
- 17.5 Bibliographical Notes
- 17.6 Exercises

Parallel K-means clustering

■ *Data parallelism* of k-means

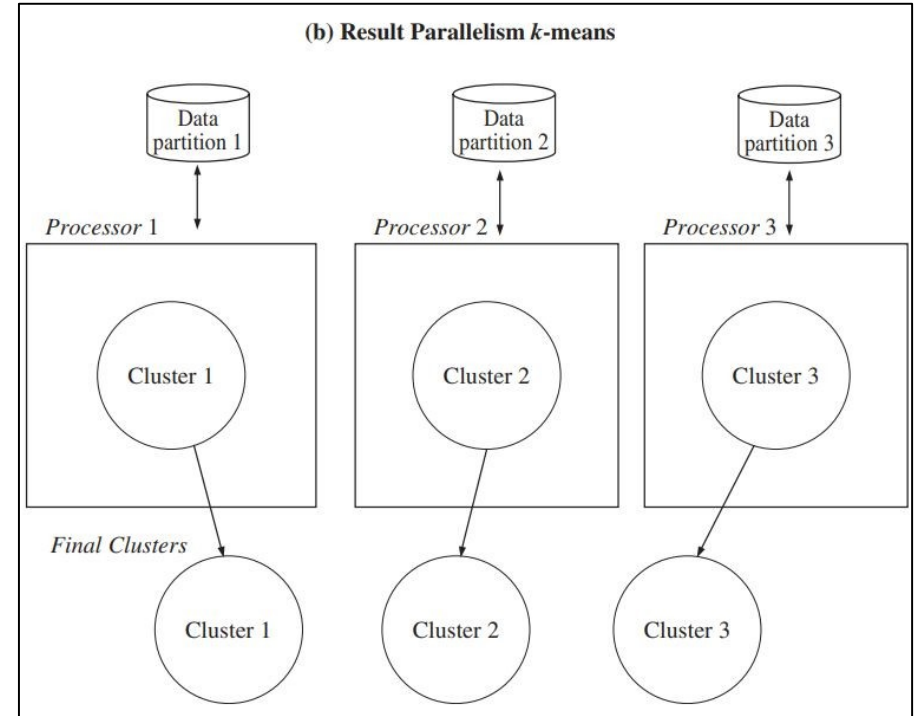
- ❑ Data is partitioned into multiple partition
- ❑ Each processor will work independently to create three clusters
- ❑ The final clusters from each processor are respectively united



Parallel K-means clustering

■ **Result Parallelism** of k-means

- ❑ Focuses on clusters partitioning
- ❑ Each processor will work on a particular target cluster
- ❑ During the iteration, the memberships of cluster can change. - ❑ data movement across processors



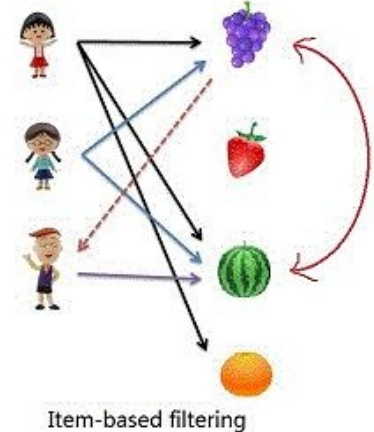
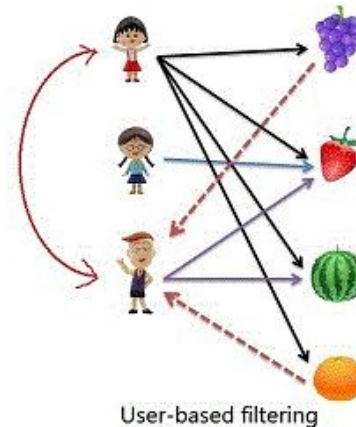
Collaborative Filtering – User based

It calculates the **similarity between users** to make implicit recommendation.

Steps:

1. Calculate the similarity between *PersonA* and all other users.
2. Predict the ratings of items for *PersonA* based on similar users.
3. Select top-N rated items for *PersonA*.

Similar
users



Flux Quiz 11

Collaboration Filtering: Walkthrough Example

Name	Star Trek	Star wars	Superman	Batman	Hulk
Harry	4	2	?	5	4
John	5	3	4	?	3
Rob	3	?	4	4	3

Aim: Recommend top-2 movies
to Harry

Collaboration Filtering: Walkthrough Example (user-based)

Step 1: Calculate the similarity between Harry and all other users

Name	Star Trek	Star wars	Superman	Batman	Hulk
Harry	4	2	?	5	4
John	5	3	4	?	3
Rob	3	?	4	4	3

Cosine similarity

$$\text{sim}(u, u') = \cos(\theta) = \frac{\mathbf{r}_u \cdot \mathbf{r}_{u'}}{\|\mathbf{r}_u\| \|\mathbf{r}_{u'}\|} = \sum_i \frac{r_{ui} r_{u'i}}{\sqrt{\sum_i r_{ui}^2} \sqrt{\sum_i r_{u'i}^2}}$$

r_{ui} - value of ratings user u gives to item i

Collaboration Filtering: Walkthrough Example (user-based)

Step 1: Calculate the similarity between Harry and all other users

Name	Star Trek	Star wars	Superman	Batman	Hulk
Harry	4	2	?	5	4
John	5	3	4	?	3
Rob	3	?	4	4	3

$$s(\theta) = \text{sim}(u, u') = \cos(\theta) = \frac{\mathbf{r}_u \cdot \mathbf{r}_{u'}}{\|\mathbf{r}_u\| \|\mathbf{r}_{u'}\|} = \sum$$

r_{ui} - value of ratings user u gives to item i

Cosine similarity

$$\begin{aligned}\text{Sim}(\text{Harry, John}) &= \frac{(4*5)+(2*3)+(4*3)}{\text{sqrt}(4^2+2^2+4^2)*\text{sqrt}(5^2+3^2+3^2)} \\ &= 0.97\end{aligned}$$

$$\begin{aligned}\text{Sim}(\text{Harry, Rob}) &= \frac{(4*3)+(5*4)+(4*3)}{\text{sqrt}(4^2+5^2+4^2)*\text{sqrt}(3^2+4^2+3^2)} \\ &= 1.00\end{aligned}$$

Collaboration Filtering: Walkthrough Example (user-based)

Step 2: Predict the ratings of movies for Harry

Name	Star Trek	Star wars	Superman	Batman	Hulk
Harry	4	2	?	5	4
John	5	3	4	?	3
Rob	3	?	4	4	3

Predicted rating is calculated based on aggregation of some similar users' rating of the item

$$r_{u,i} = k \sum_{u' \in U} \text{simil}(u, u') r_{u',i}$$

with normalising factor

$$k = 1 / \sum_{u' \in U} |\text{simil}(u, u')|,$$

Calculate k as a normalising factor $k = \frac{1}{(0.97+1)} = 0.51$

$$\begin{aligned} R(\text{Harry}, \text{Superman}) &= k * ((\text{sim}(\text{Harry}, \text{John}) * R(\text{John}, \text{Superman})) + (\text{sim}(\text{Harry}, \text{Rob}) * R(\text{Rob}, \text{Superman}))) \\ &= 0.51((0.97 * 4) + (1 * 4)) = 4.02 \end{aligned}$$

Collaboration Filtering: Walkthrough Example (user-based)

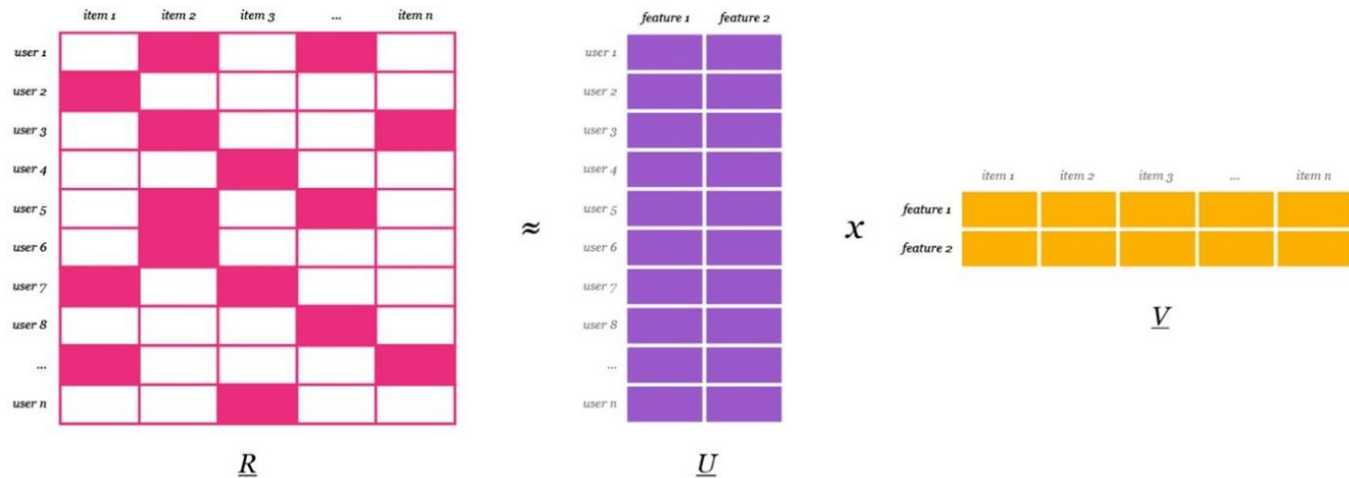
Step 3: Select top-2 rated movies for Harry

Name	Star Trek	Star wars	Superman	Batman	Hulk
Harry	4	2	4.02	5	4
John	5	3	4	?	3
Rob	3	?	4	4	3

$Top-2(Harry, movies) = \text{Batman, Superman}$

Model-based Collaborative Filtering

- ❑ Latent factor model based CF learns the (latent) user and item profiles through **matrix factorisation**
- ❑ **Matrix factorisation:** Factor a large matrix into some smaller representation of the original matrix through alternating least squares. The product of lower dimensional matrices equals the original one
- ❑ Example: Factor the rating matrix **R** into user matrix **U** and item matrix **V**



Alternating least squares

- ALS method aims to estimate the user and item factor matrices (**U** & **V**) such that their product will approximate the original rating matrix **R**.
- This is achieved by minimising root mean square error (RMSE) between the original ratings and the predicted values

ALS Procedure:

Optimizing alternately to find **U**, **V**

- Randomly initialise **U** and **V**
- Iterating the following steps:
 - Fixing **U** □ Optimising **V**
 - Fixing **V** □ Optimising **U**
- Each iteration will minimise the square

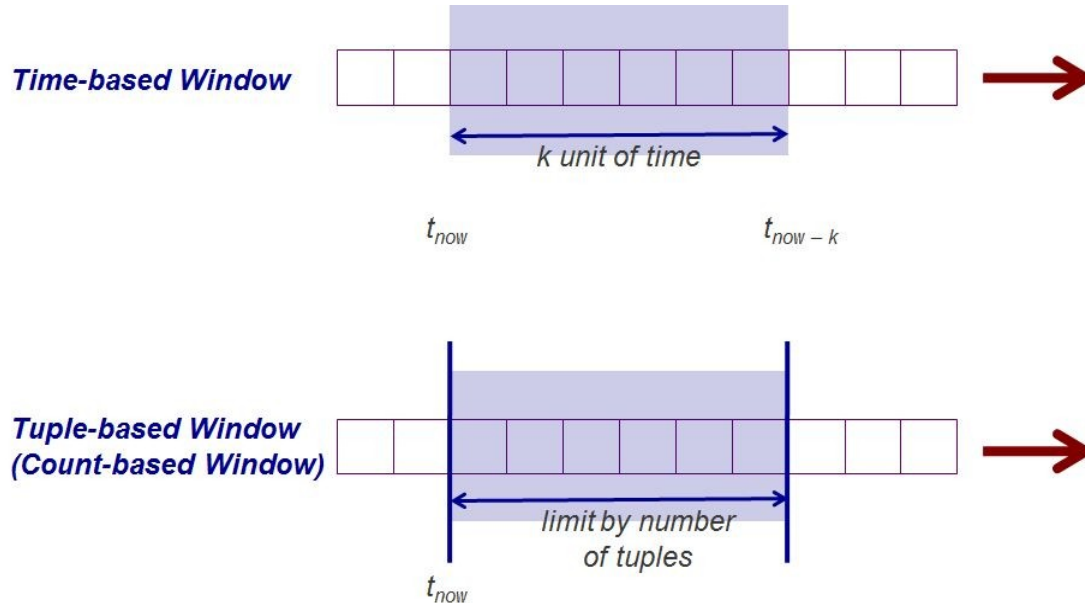


Unit Overview

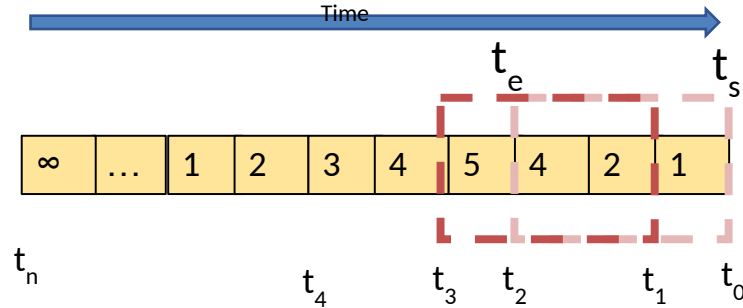
1. **Volume** □ Sessions 1, 2, 3, 4
 - How to process Big Data Volume?
2. **Complexity** □ Sessions 5, 6, 7, 8
 - How to apply machine learning algorithms to every aspect of Big Data?
3. **Velocity** □ Sessions 9, 10, 11
 - How to handle and process Fast Streaming Data?

Windowing System in Unbounded Streams

A **data stream** is a real-time, continuous, ordered (implicitly by arrival time or explicitly by timestamp) sequence of items.



Stream Window – Time Based Window

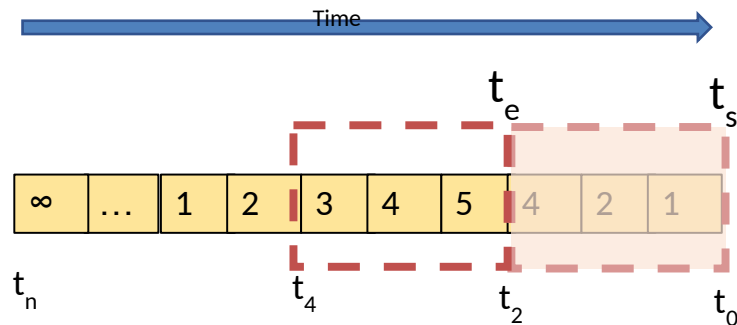


Window size 2 seconds
Slides 1 second.

- ❑ Size of window is specified by fixed time duration

Overlapping Sliding Window

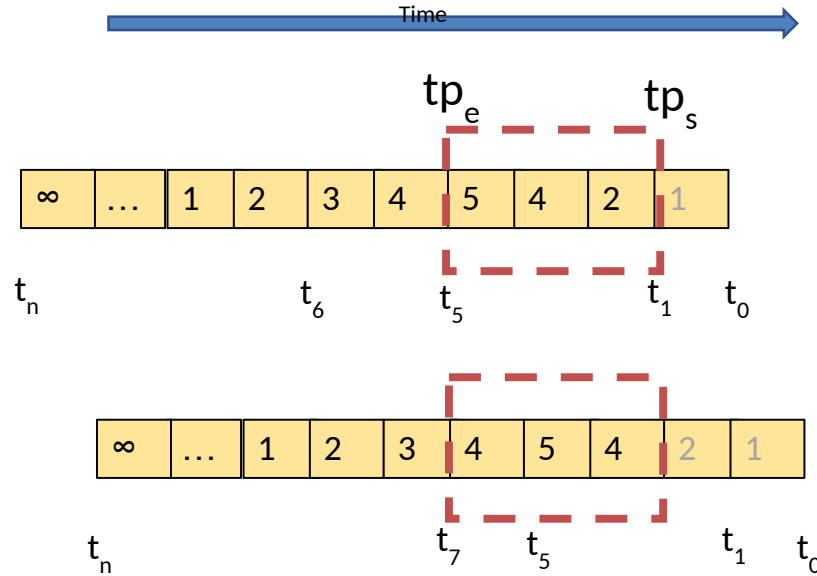
Stream Window – Time Based Window



Non-Overlapping Sliding Window

- Window size is based on time, eg 2 seconds
- Window can be advanced by:
 - $t_e - t_s$ (window size)
 - Duration less than the window size (sliding window).
- In uniform data rate, the number of tuples will be the same for each window.

Stream Window – Tuple Based Window



Window size is 3 tuples
Slide window by 1 tuple

- Size of window is specified by number of tuples in the window

Database vs Stream Processing

- Bounded data (assess to entire data).
- Relatively static data
- Complex, ad-hoc query
- Possible to backtrack during processing
- Exact answer to a query
- Tuples arrival rate is low

- Unbounded data (data keeps coming without end).
- Dynamic data.
- Simple, continuous query
- No backtracking, single pass operation.
- Approximate answer to a query
- Tuples arrival rate is high

Sliding Window: Produce an approximate answer to a data stream query is to evaluate the query not over the entire past history of data streams, but rather only over sliding windows of recent data from the streams.

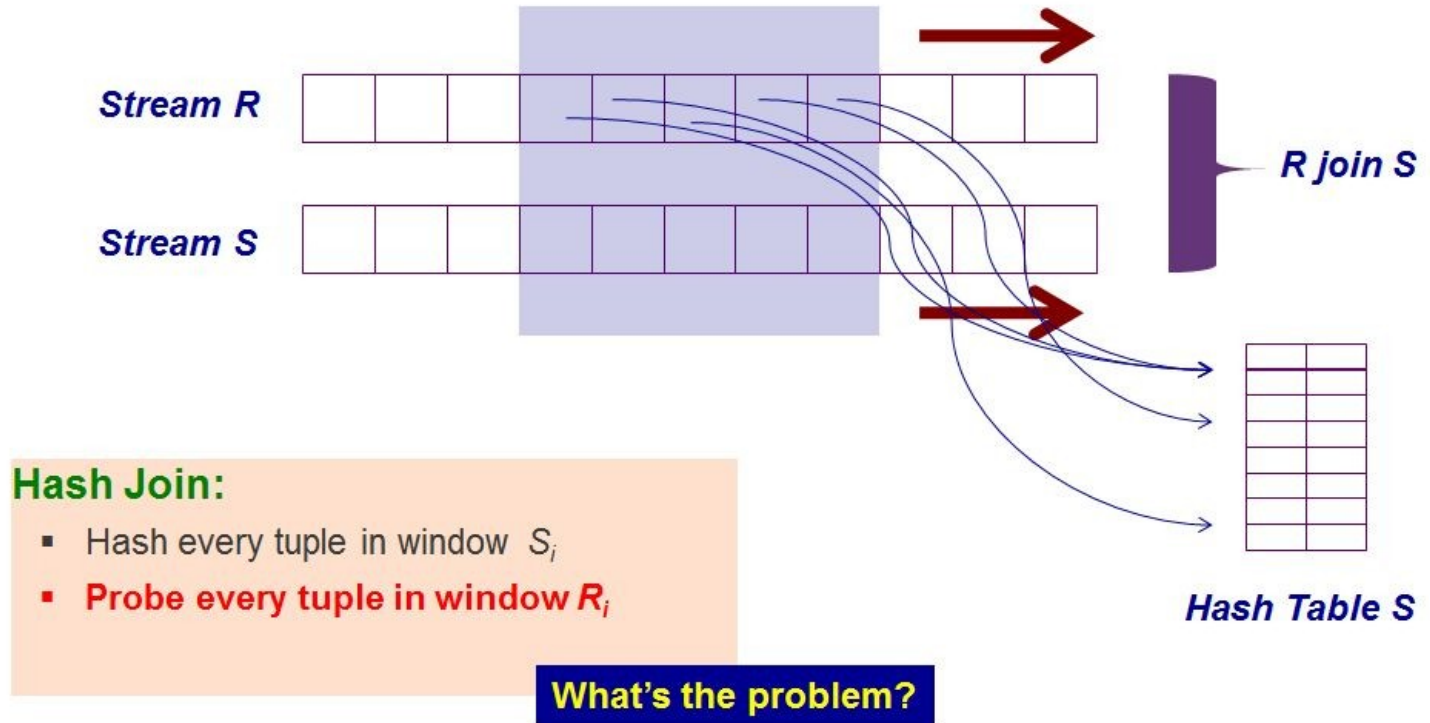
Event vs Processing time

- Event time: the time when the data is produced by the source.
- Processing time: the time when data is arrived at the processing server.
- In ideal situation, event time = processing time.
- In real world event time is earlier than the processing time due to network delay.
- The delay can be uniformed (ideal situation) or non-uniform (most of real network situation).
- Data may arrive in “burst” (bursty network).

Joins In Data Streams

- Symmetric Hash Join
- M Join
- AM Join
- Handshake Join

Hash Join



Problem: If r comes in first and then s comes in later, then r will not be compared with the s

- If r and s has same key, we will miss join operation

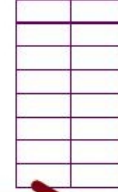
Symmetric Hash Join

Step 1:

tuple r arrives

r

Stream R

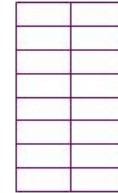


Hash Table R

s

tuple s not yet
processed

Stream S



Hash Table S



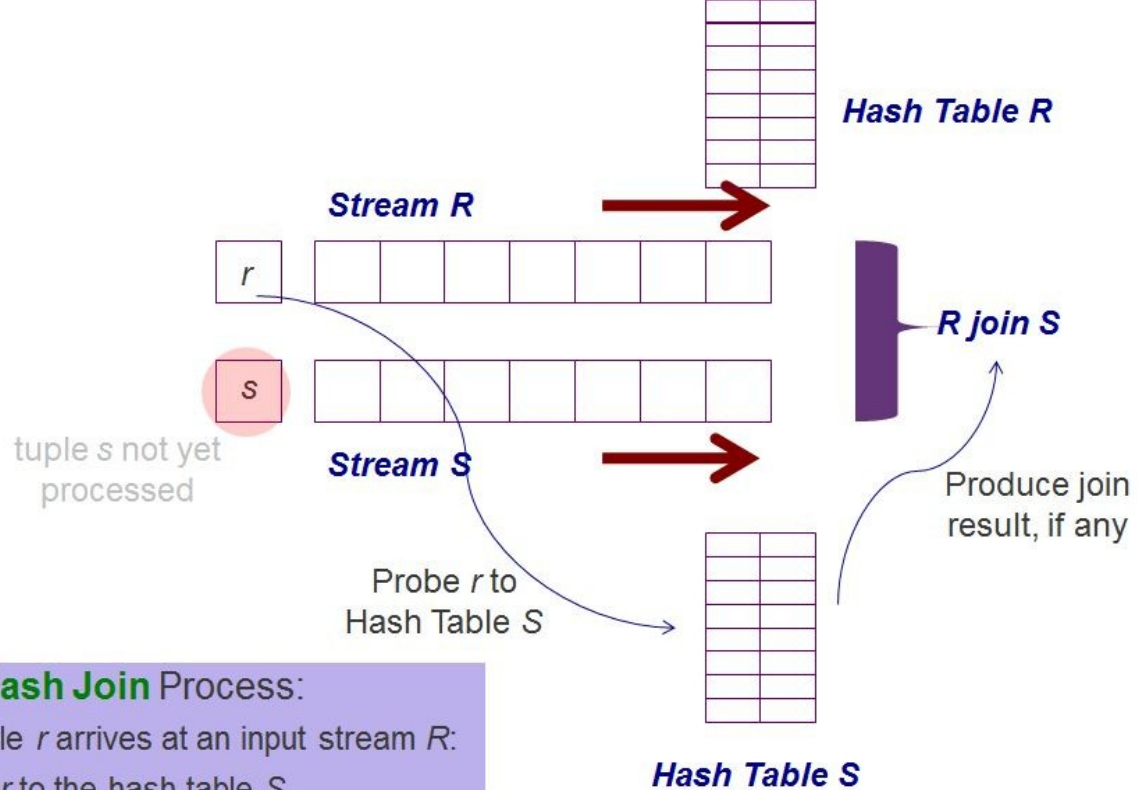
$R \text{ join } S$

Symmetric Hash Join Process:

- When a tuple r arrives at an input stream R :
 - Probe r to the hash table S
 - Hash tuple r into hash table R
 - Insert new tuple r into stream R

Symmetric Hash Join

Step 2:

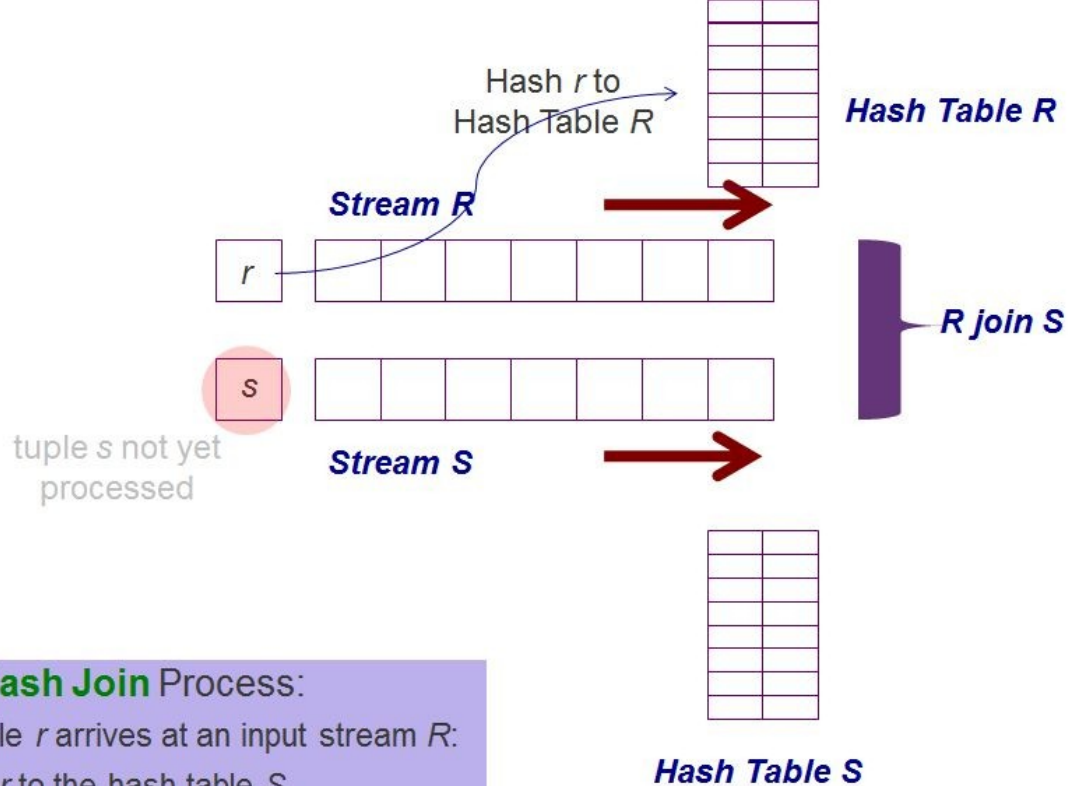


Symmetric Hash Join Process:

- When a tuple *r* arrives at an input stream *R*:
 - Probe *r* to the hash table *S*
 - Hash tuple *r* into hash table *R*
 - Insert new tuple *r* into stream *R*

Symmetric Hash Join

Step 3:

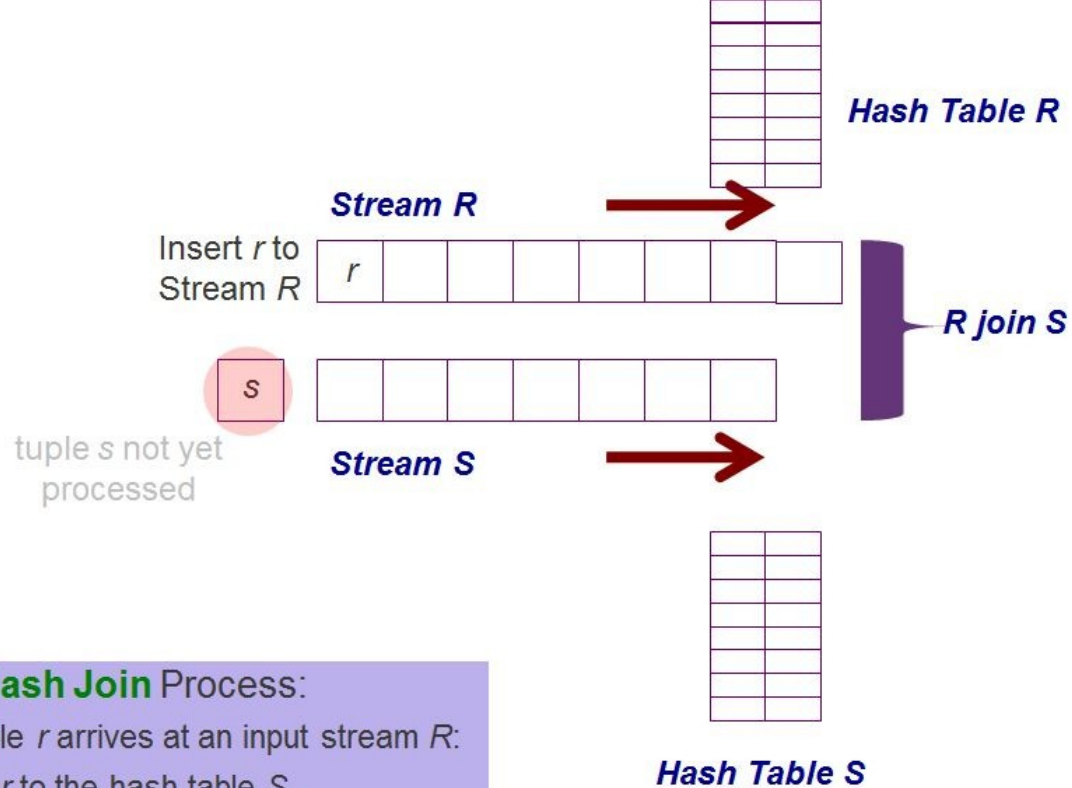


Symmetric Hash Join Process:

- When a tuple *r* arrives at an input stream *R*:
 - Probe *r* to the hash table *S*
 - Hash tuple *r* into hash table *R*
 - Insert new tuple *r* into stream *R*

Symmetric Hash Join

Step 4:

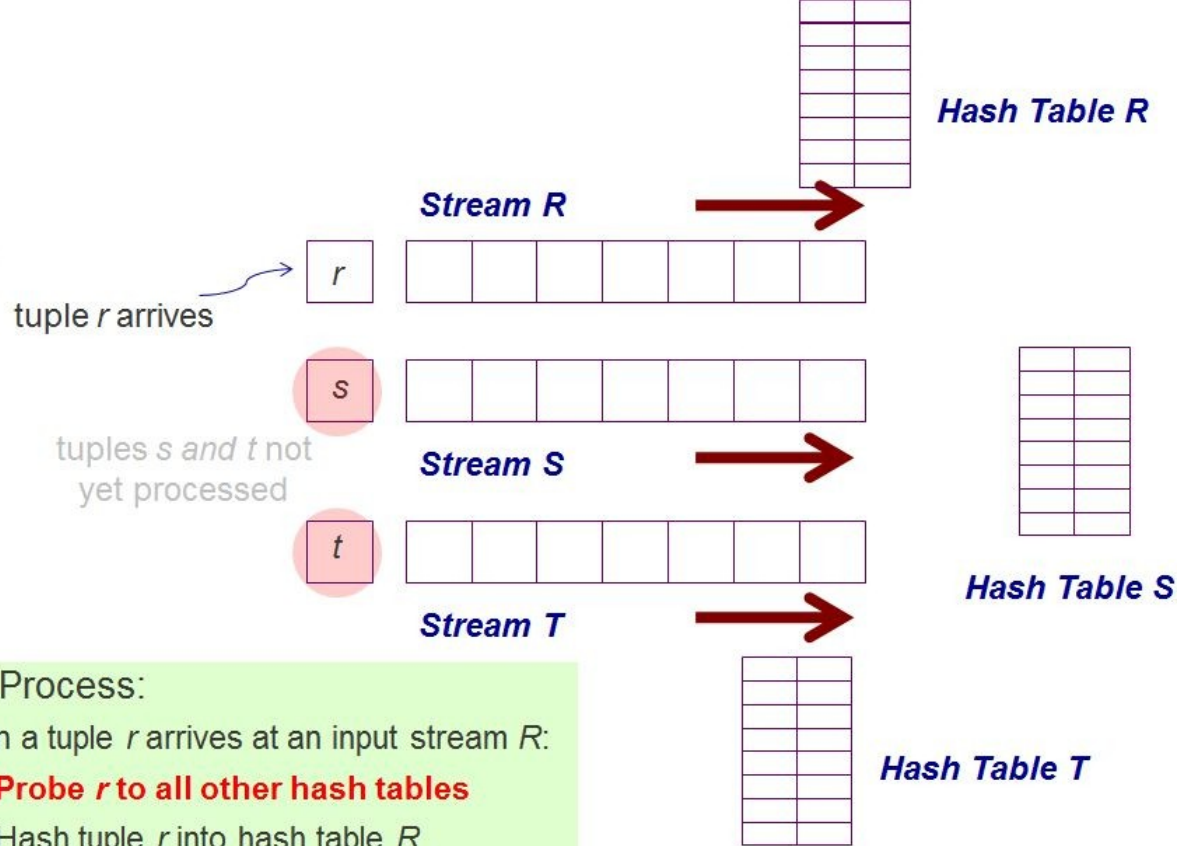


Symmetric Hash Join Process:

- When a tuple r arrives at an input stream R :
 - Probe r to the hash table S
 - Hash tuple r into hash table R
 - Insert new tuple r into stream R

M-Join

Step 1:

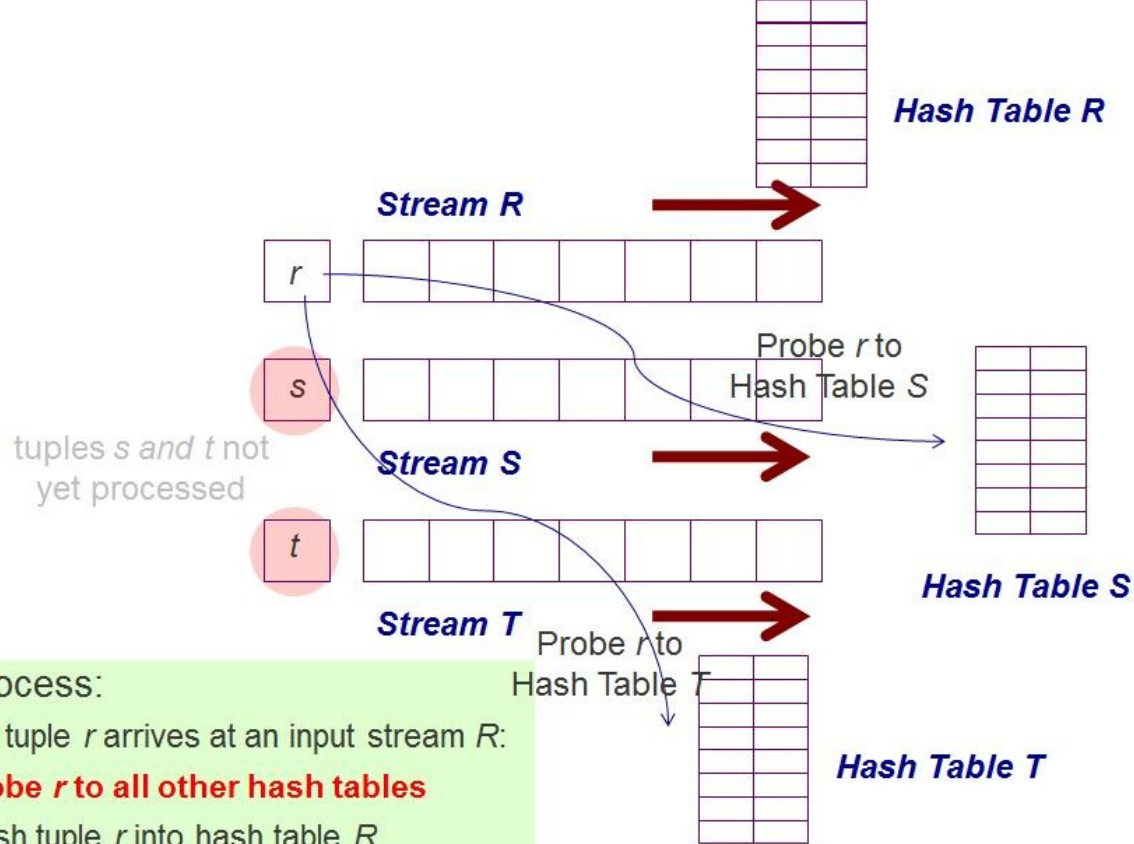


M-Join Process:

- When a tuple *r* arrives at an input stream *R*:
 - **Probe *r* to all other hash tables**
 - Hash tuple *r* into hash table *R*
 - Insert new tuple *r* into stream *R*

M-Join

Step 2:

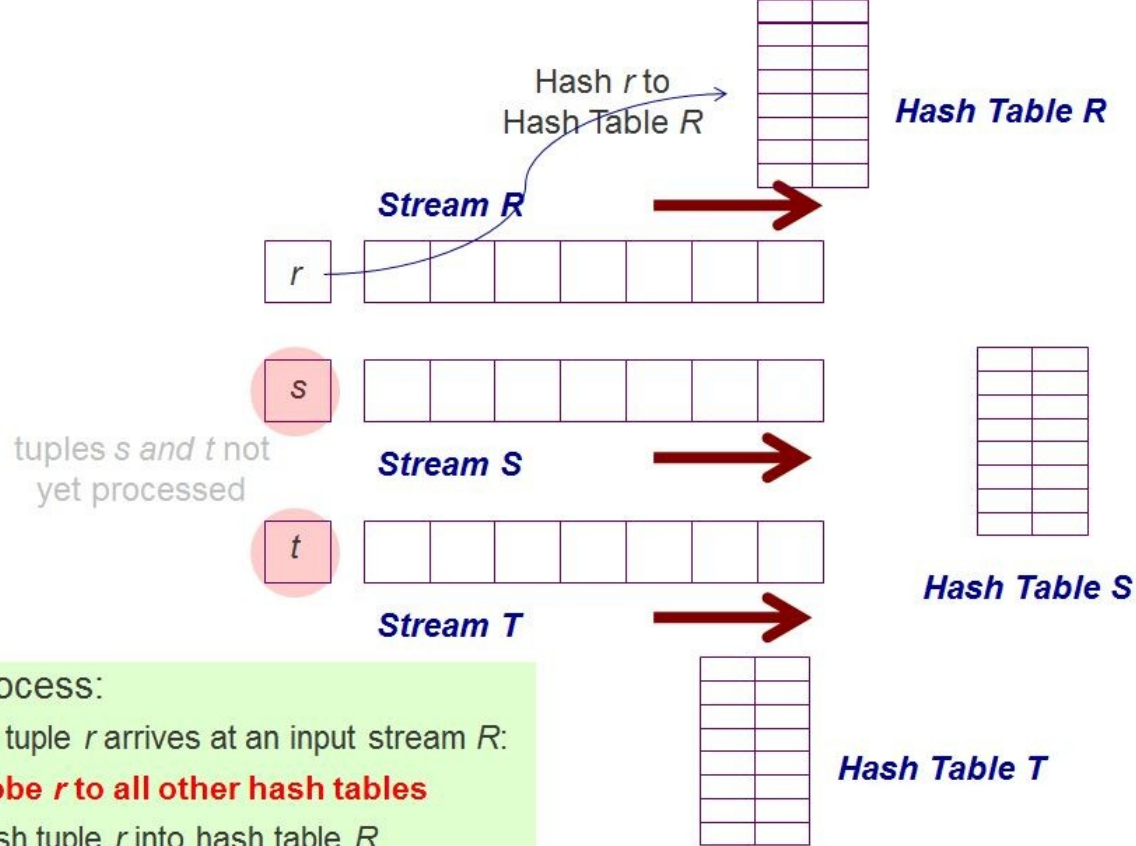


M-Join Process:

- When a tuple *r* arrives at an input stream *R*:
 - **Probe *r* to all other hash tables**
 - Hash tuple *r* into hash table *R*
 - Insert new tuple *r* into stream *R*

M-Join

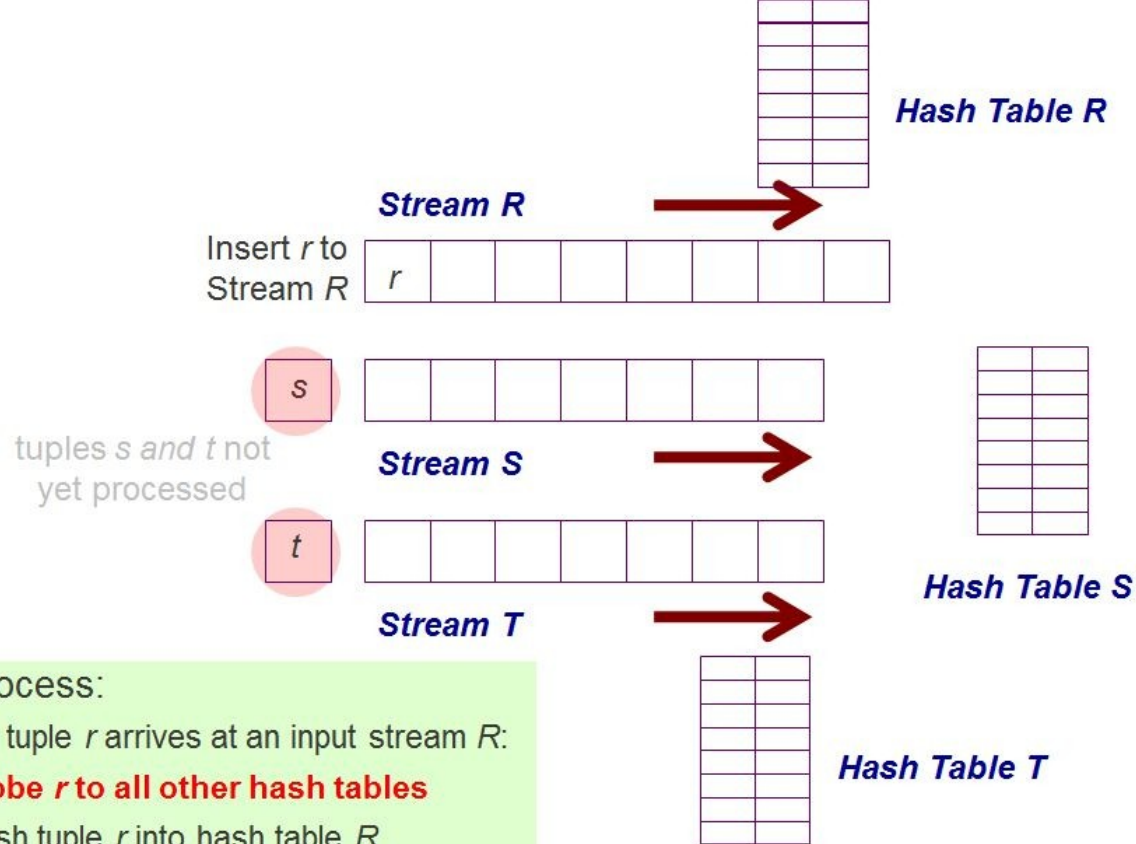
Step 3:



M-Join Process:

- When a tuple r arrives at an input stream R :
 - Probe r to all other hash tables**
 - Hash tuple r into hash table R
 - Insert new tuple r into stream R

Step 3:



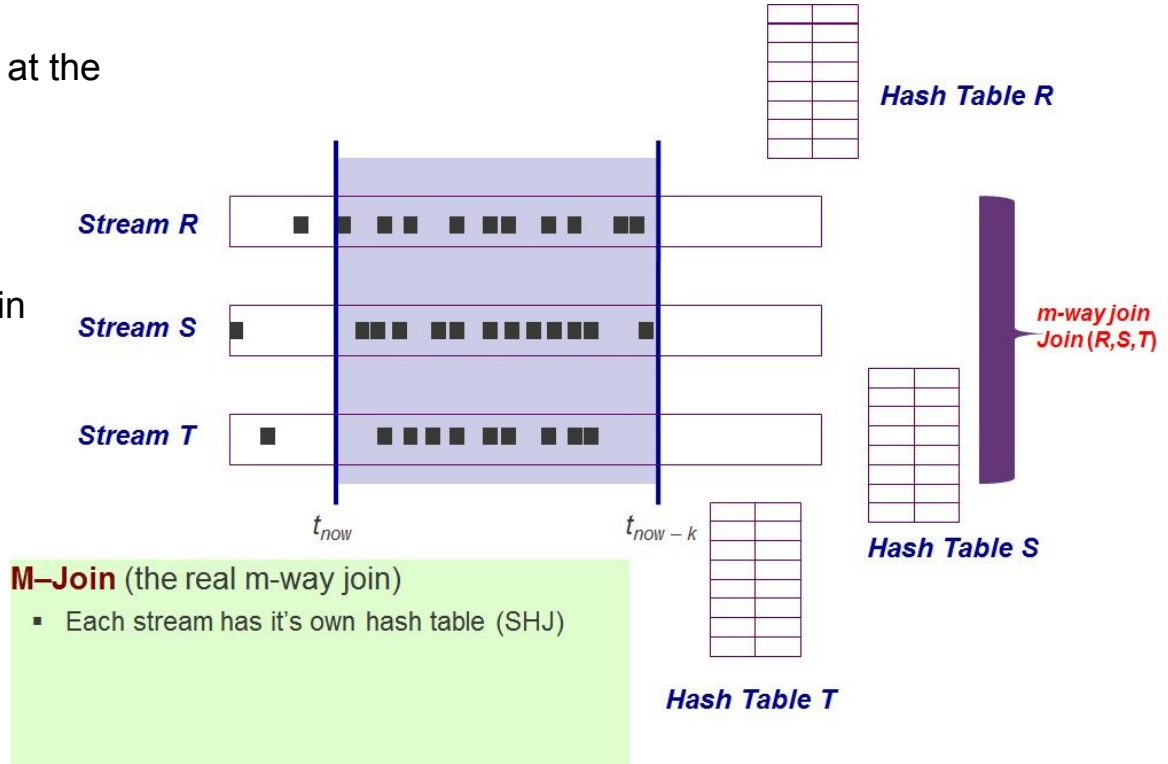
M-Join Process:

- When a tuple r arrives at an input stream R :
 - **Probe r to all other hash tables**
 - Hash tuple r into hash table R
 - Insert new tuple r into stream R

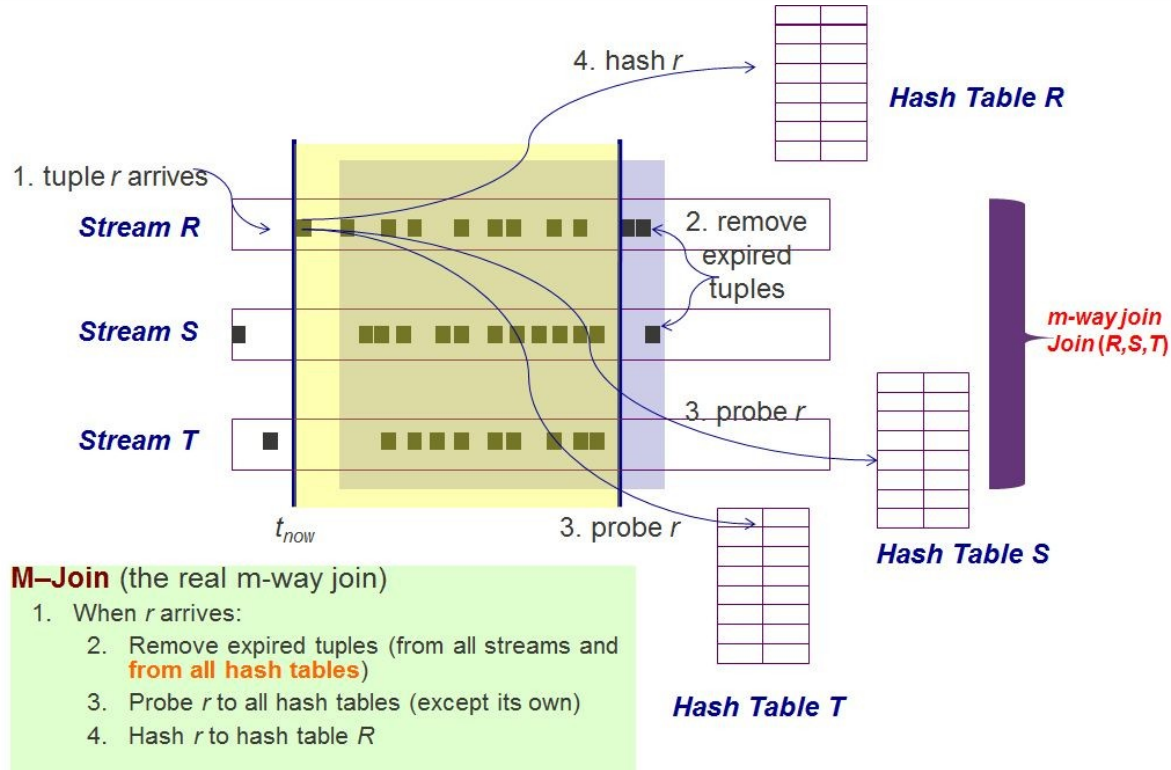
Tuple Slide (Using M-Join)

How to join more than two streams at the same time?

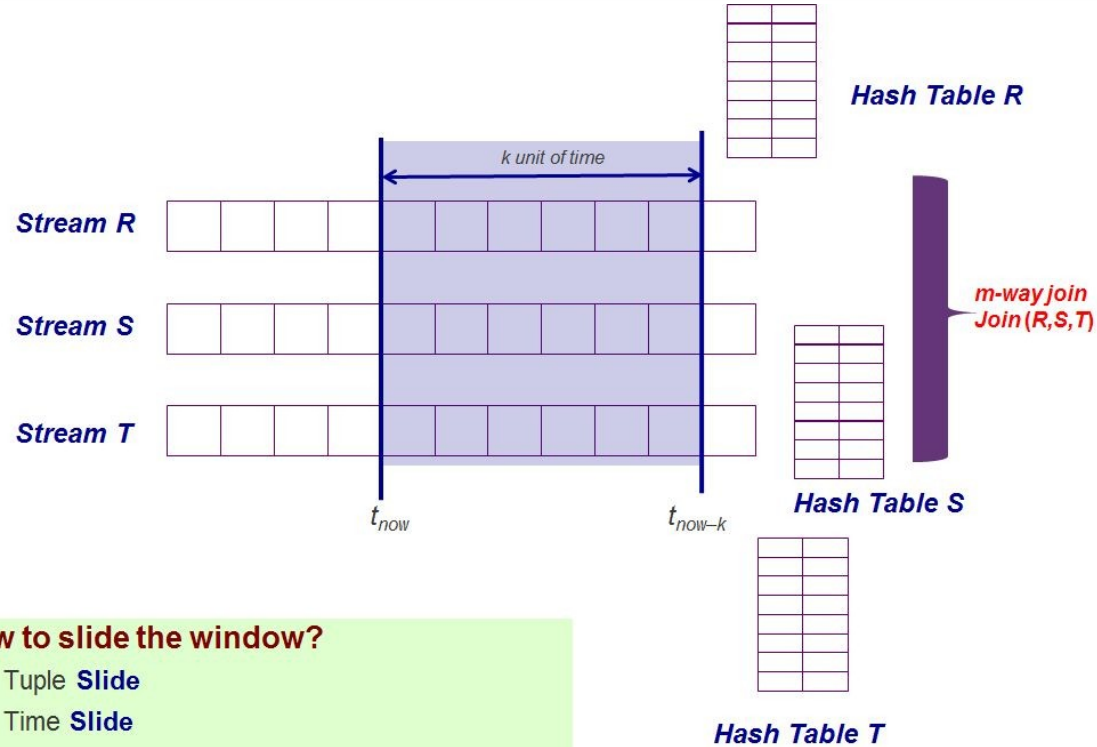
- ❑ Use **M-Join** – a multiway streaming join
- ❑ Based on symmetric hash join (work similarly to two-stream)



Tuple Slide (Using M-Join)



Time Slide (Using M-Join)



How to slide the window?

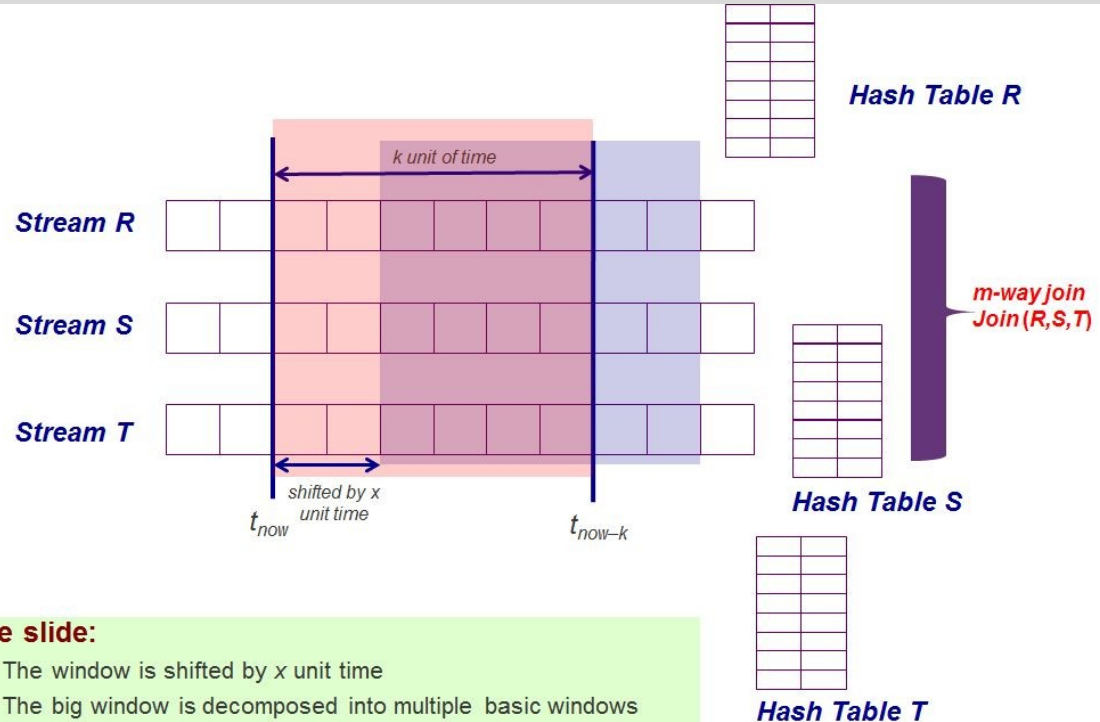
- Tuple **Slide**
- Time **Slide**

Time Slide (Using M-Join)

- Let one box here represents one **unit time** or one **basic window**.

Ex: 1 unit time = 1 minute

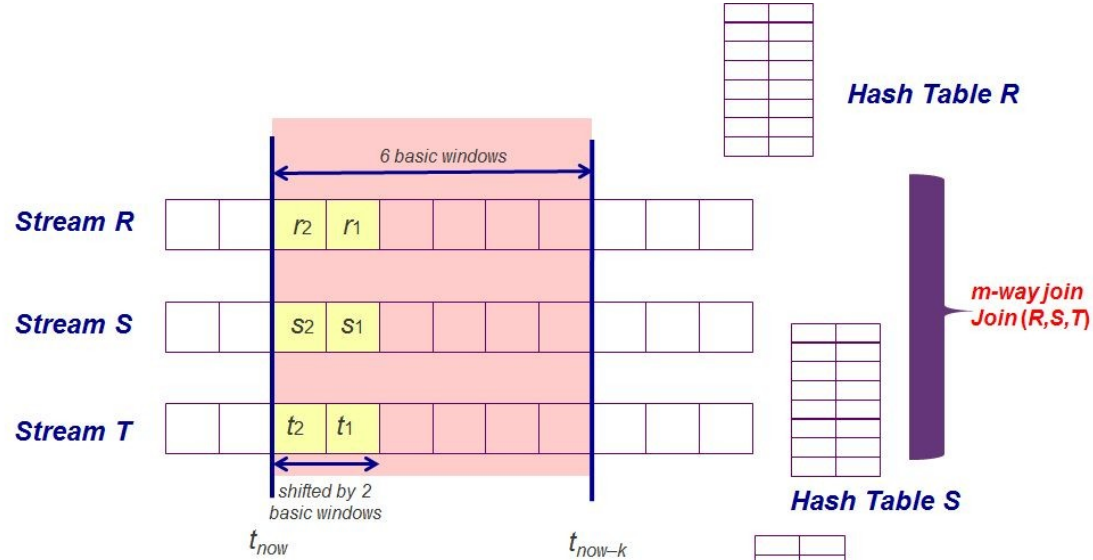
- Window size = 6 mins
- slide = 2mins



Time slide:

- The window is shifted by x unit time
- The big window is decomposed into multiple basic windows
- The big window is then **shifted by 1 or more basic windows**

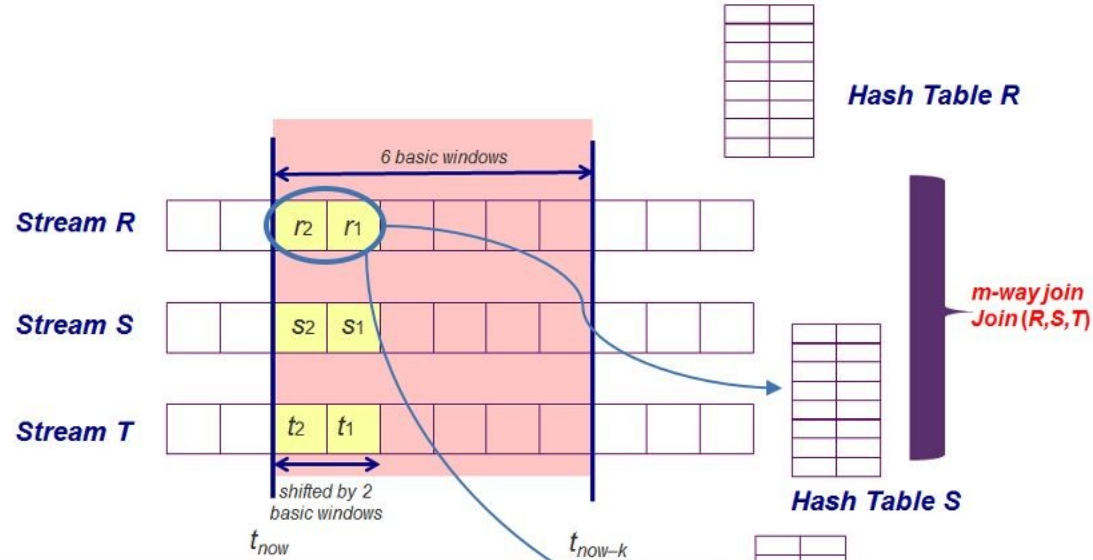
Time Slide (Using M-Join)



Time Slide (using M-Join):

- We use the **current window** (assume the previous expired basic windows have been removed as part of the previous join process)
- In the above example, the latest 2 basic windows are new basic windows
- Note that the **hash tables do not yet have tuples from these 2 basic windows**

Time Slide (Using M-Join)

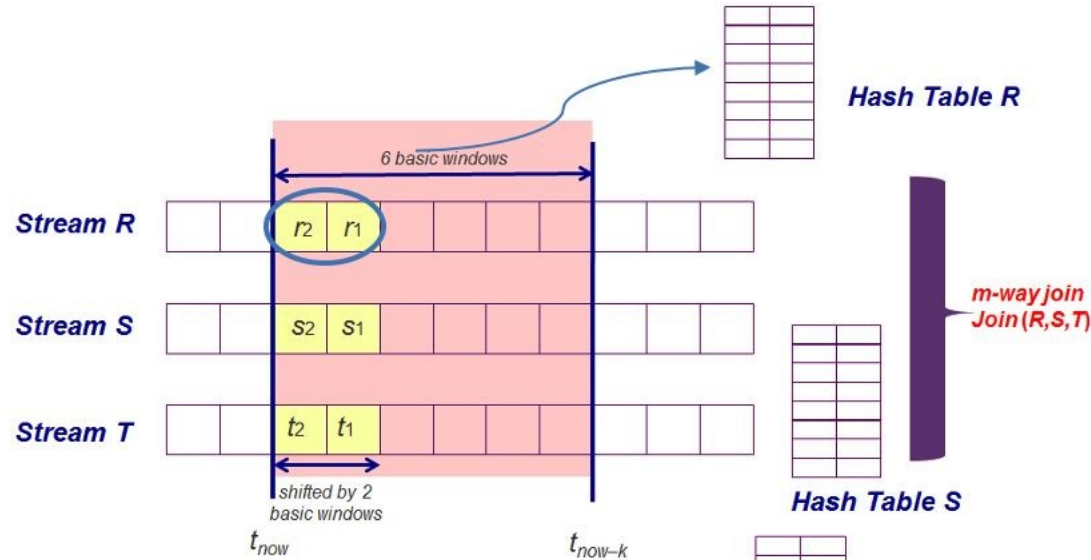


Time Slide (using M-Join):

▪ Process Stream R:

- Take all tuples from basic windows r_1 and r_2 , and probe to hash tables S and T
- Take all tuples from basic windows r_1 and r_2 to hash table R

Time Slide (Using M-Join)

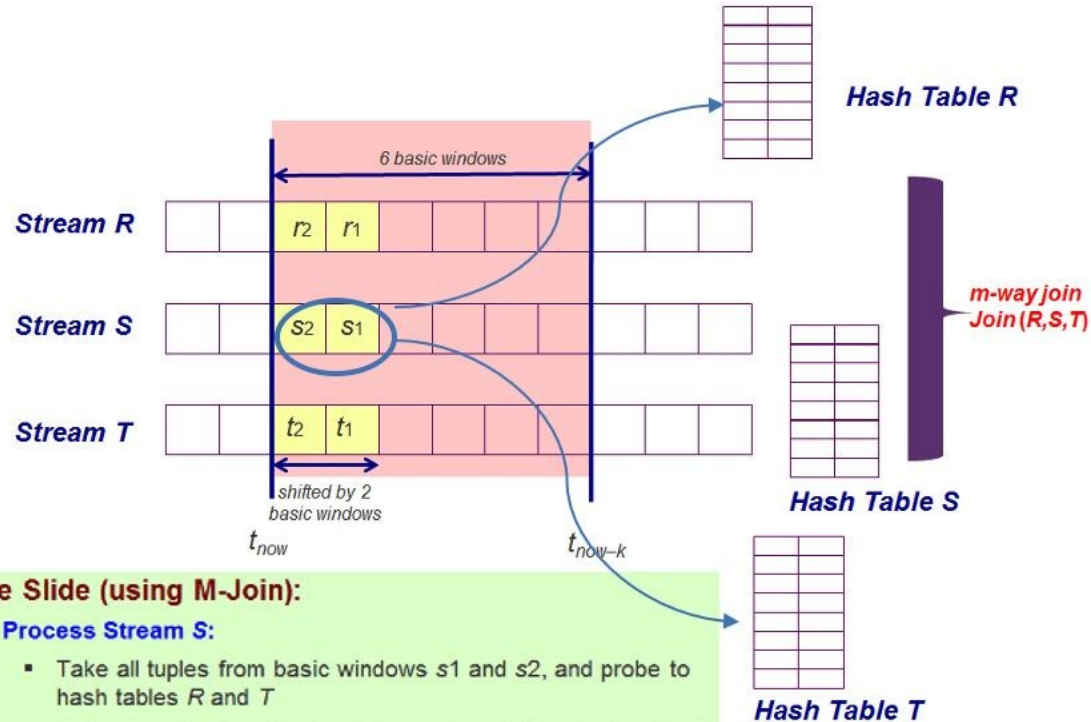


Time Slide (using M-Join):

▪ Process Stream R:

- Take all tuples from basic windows r_1 and r_2 , and probe to hash tables S and T
- Take all tuples from basic windows r_1 and r_2 to hash table R

Time Slide (Using M-Join)

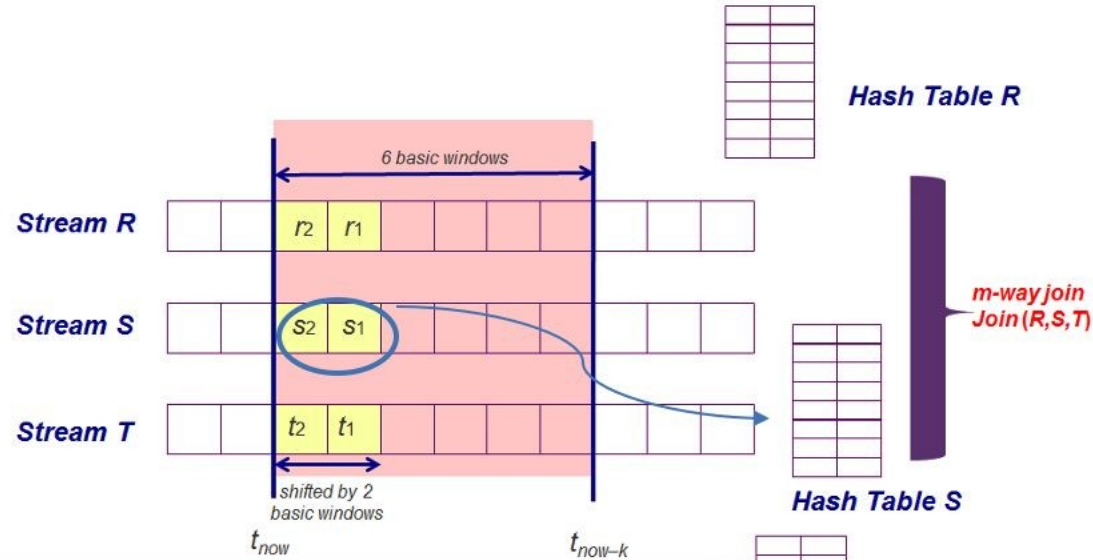


Time Slide (using M-Join):

▪ Process Stream S:

- Take all tuples from basic windows s_1 and s_2 , and probe to hash tables R and T
- Take all tuples from basic windows s_1 and s_2 to hash table S

Time Slide (Using M-Join)

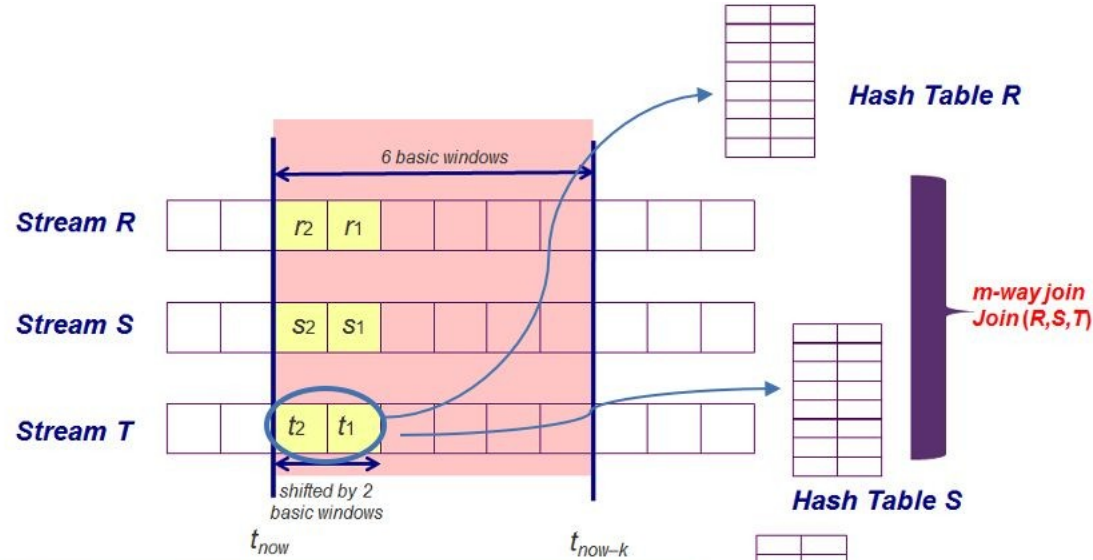


Time Slide (using M-Join):

▪ Process Stream S:

- Take all tuples from basic windows s_1 and s_2 , and probe to hash tables R and T
- Take all tuples from basic windows s_1 and s_2 to hash table S

Time Slide (Using M-Join)

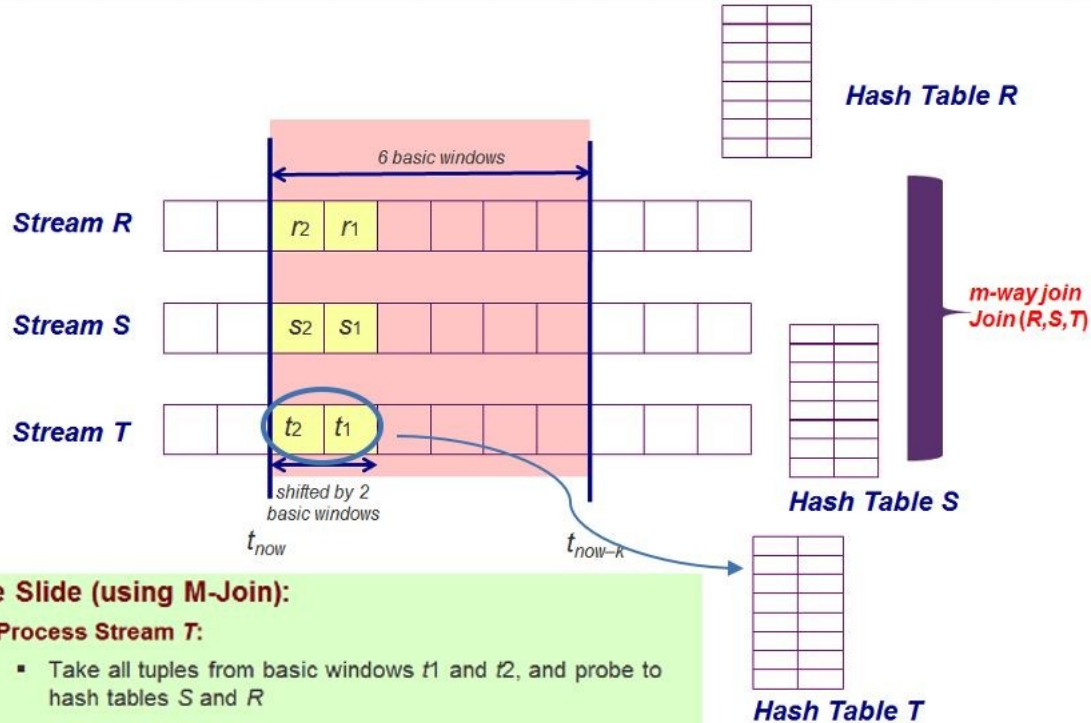


Time Slide (using M-Join):

▪ Process Stream T:

- Take all tuples from basic windows t_1 and t_2 , and probe to hash tables S and R
- Take all tuples from basic windows t_1 and t_2 to hash table T

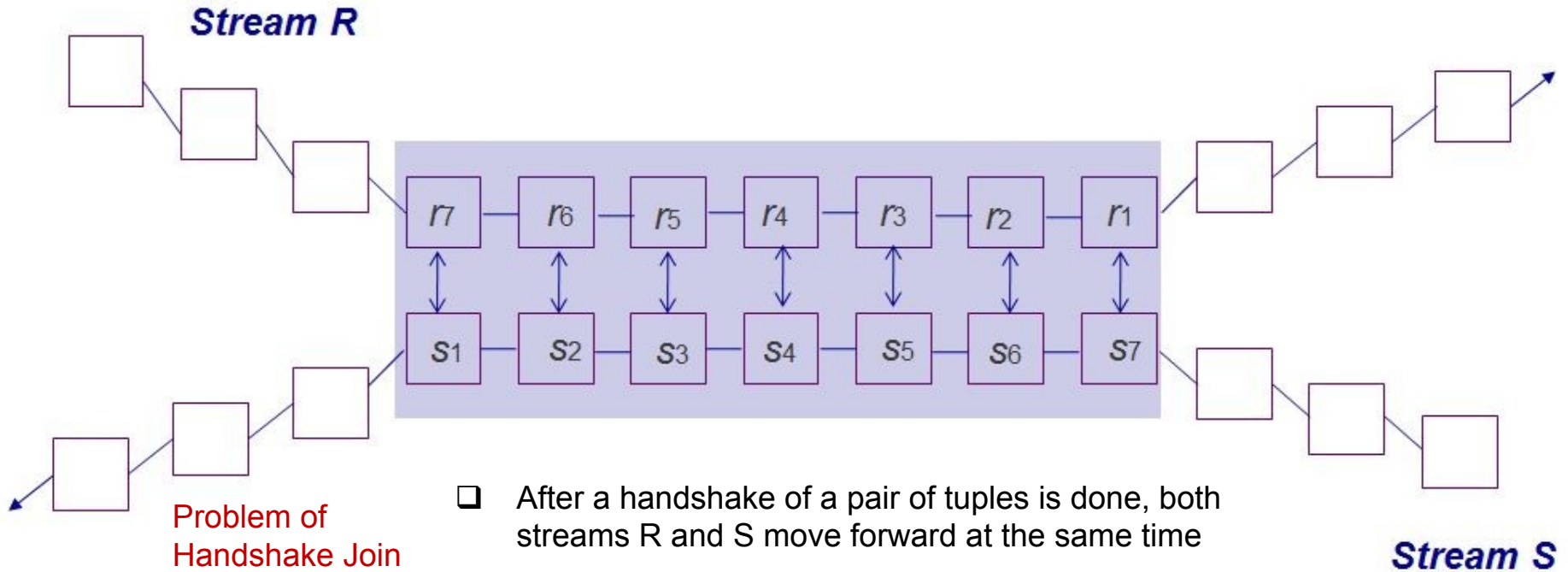
Time Slide (Using M-Join)



Time Slide (using M-Join):

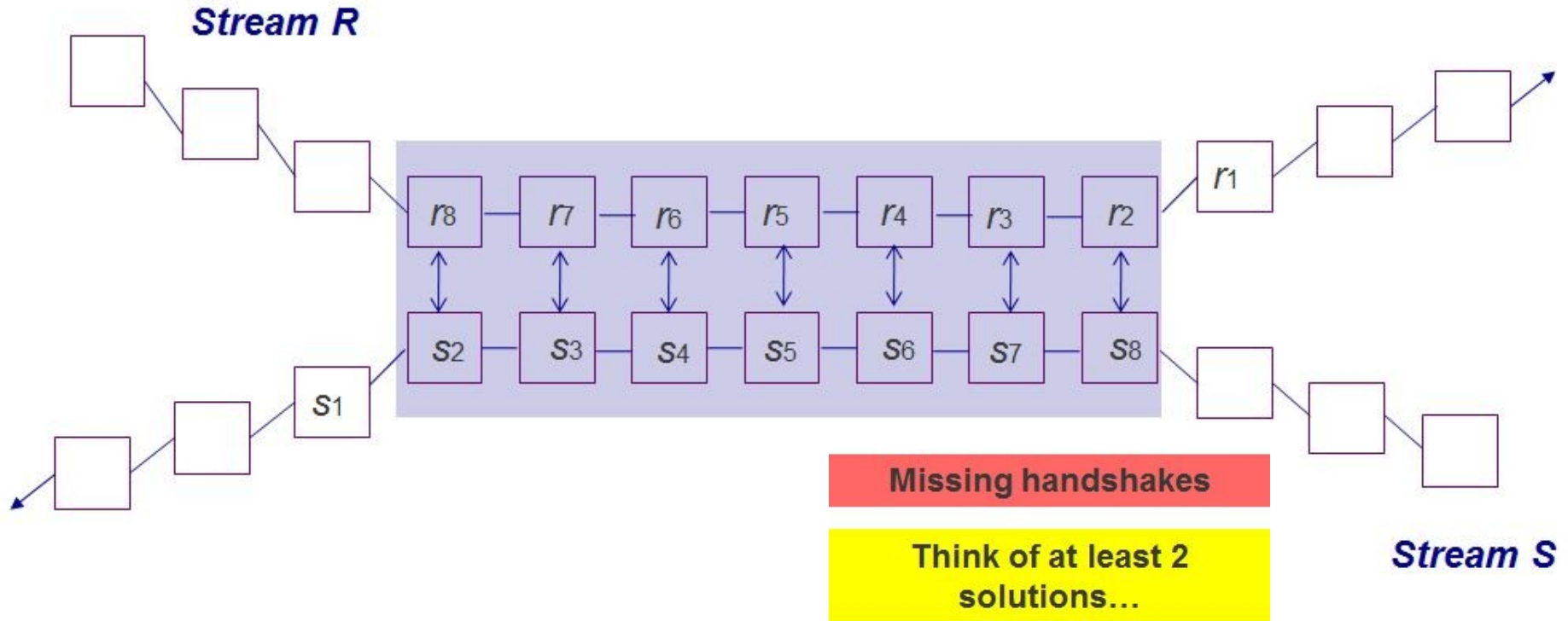
- **Process Stream T:**
 - Take all tuples from basic windows t_1 and t_2 , and probe to hash tables S and R
 - Take all tuples from basic windows t_1 and t_2 to hash table T

Handshake Join



- ❑ After a handshake of a pair of tuples is done, both streams R and S move forward at the same time
- ❑ As a result, we will miss a handshake because one tuple from R moves to the right, and one tuple from S moves to the left, and one handshake will be missed

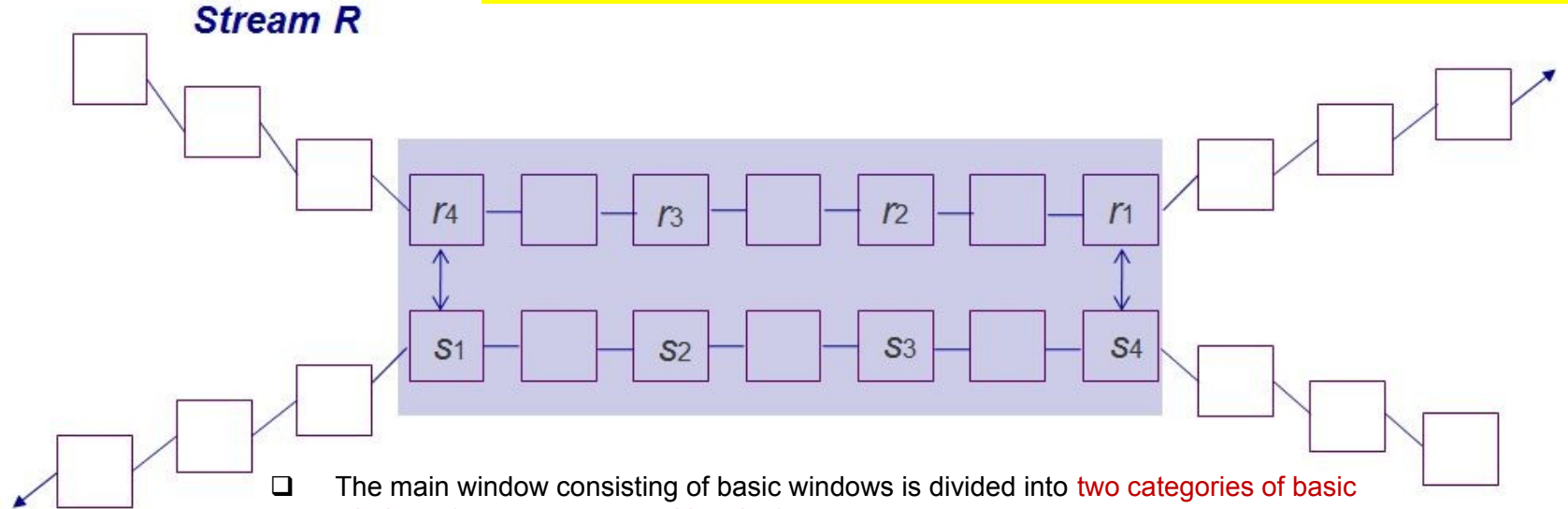
Handshake Join



Handshake Join (Solution 1)

Solution 1:

Main Idea: Alternating basic window must be left empty in the stream.



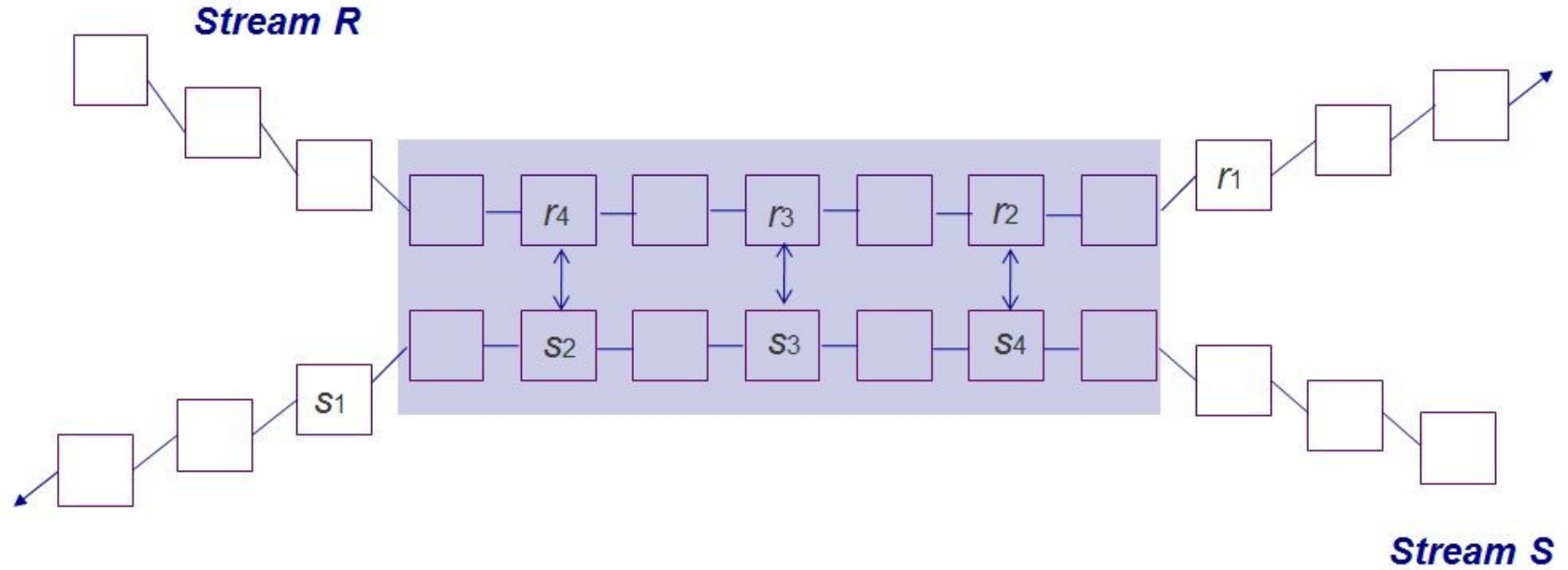
- ❑ The main window consisting of basic windows is divided into **two categories of basic windows (one empty, one with value)**.

- ❑ Each basic window will be used in an alternate fashion. When the streams move, they will move to the empty basic window, and hence no missing handshake will occur.

Stream S

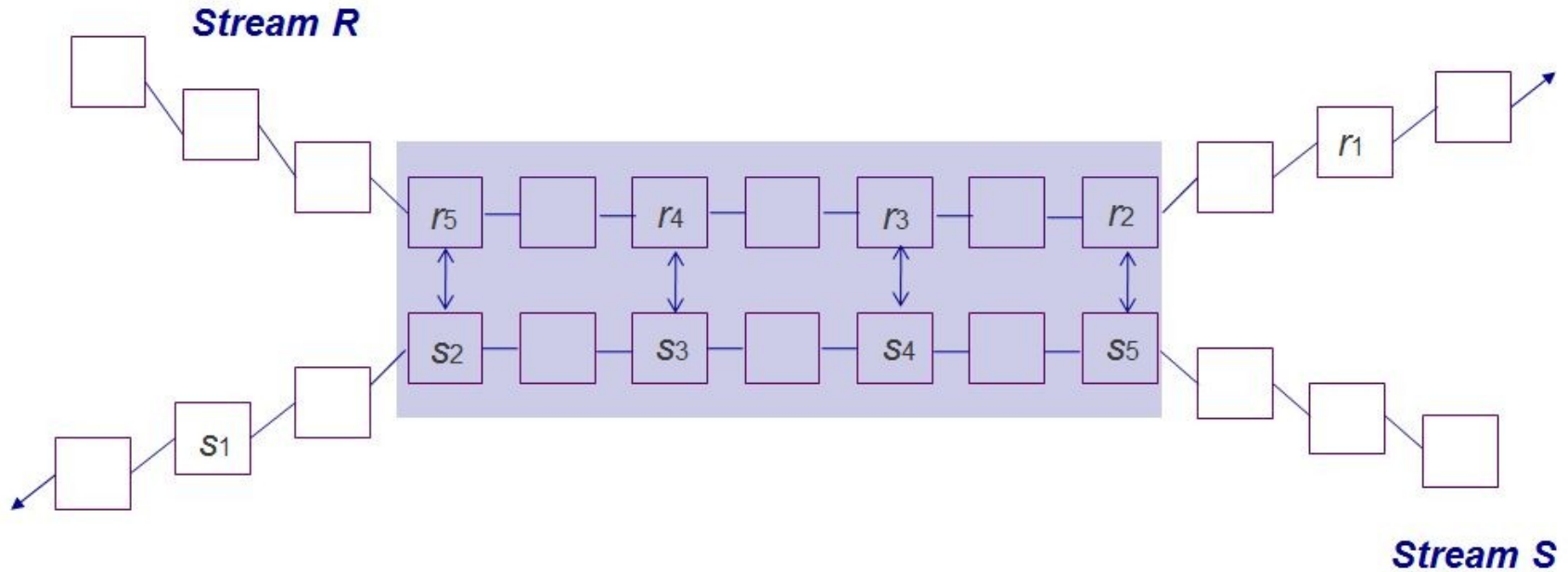
Handshake Join (Solution 1)

Solution 1:



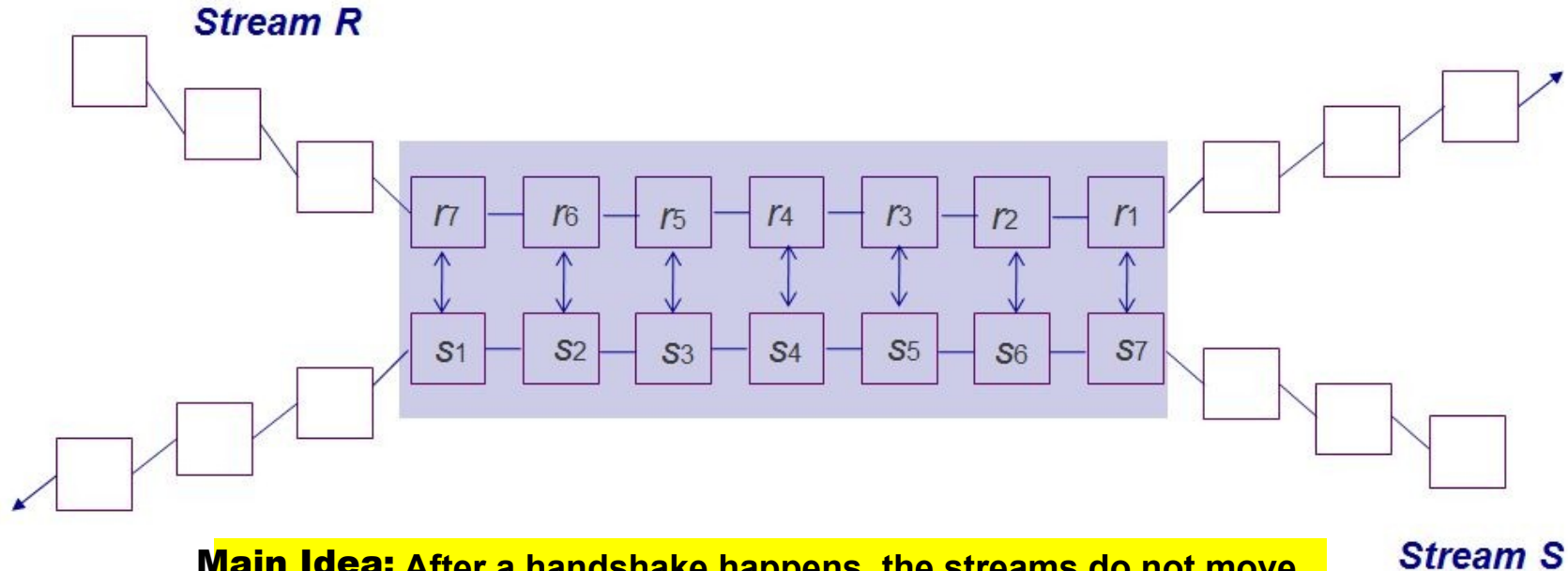
Handshake Join (Solution 1)

Solution 1:



Handshake Join (Solution 2)

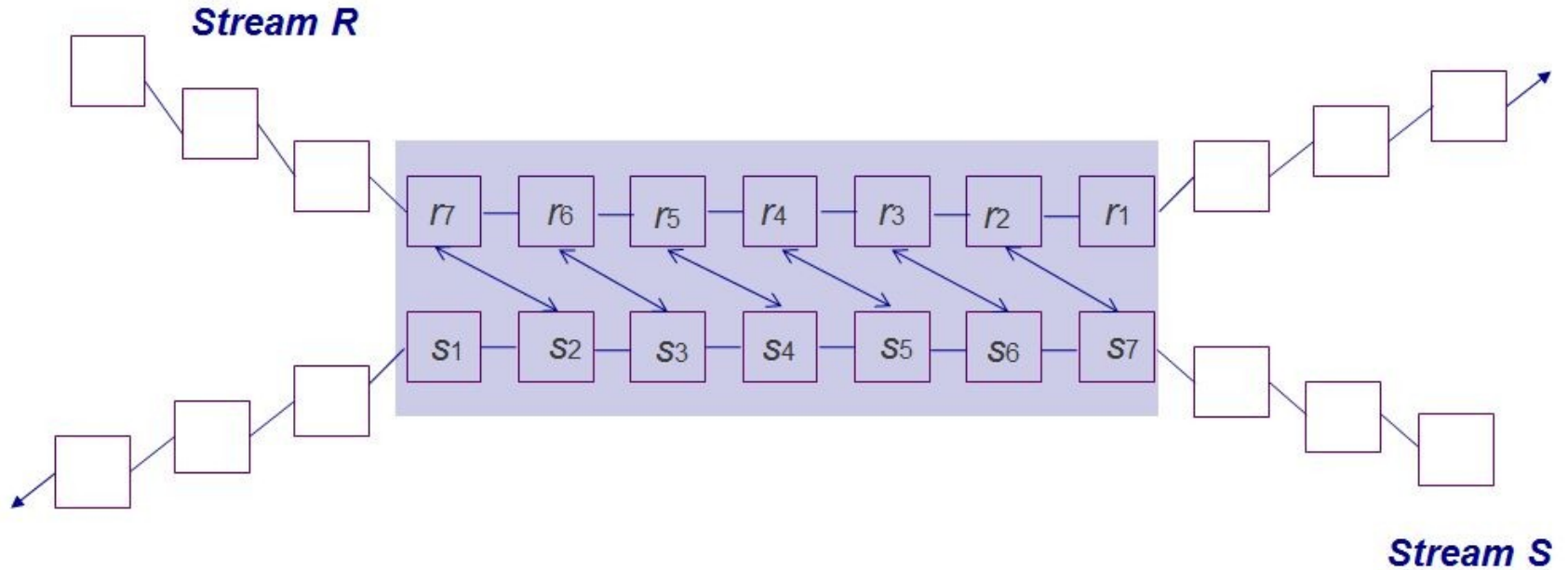
Solution 2:



Main Idea: After a handshake happens, the streams do not move forward, but each tuple in the stream performs a handshake with the next tuple, and then after that both streams move forward.

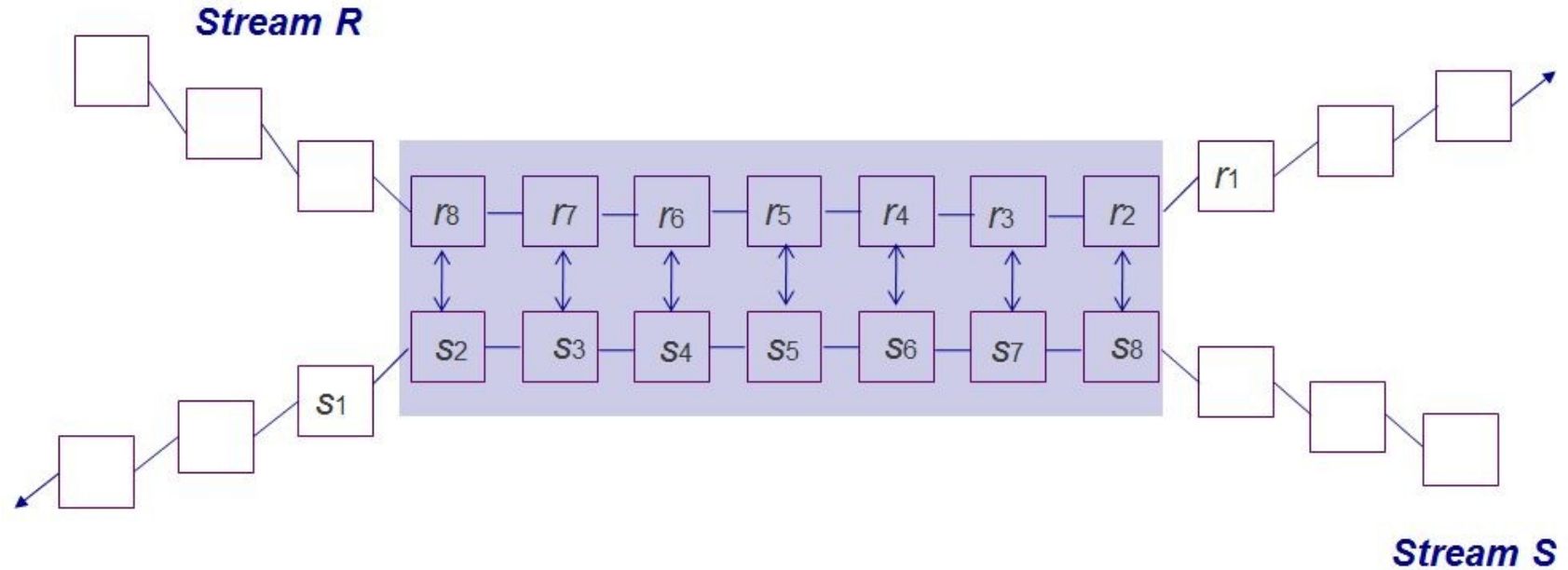
Handshake Join (Solution 2)

Solution 2:



Handshake Join (Solution 2)

Solution 2:

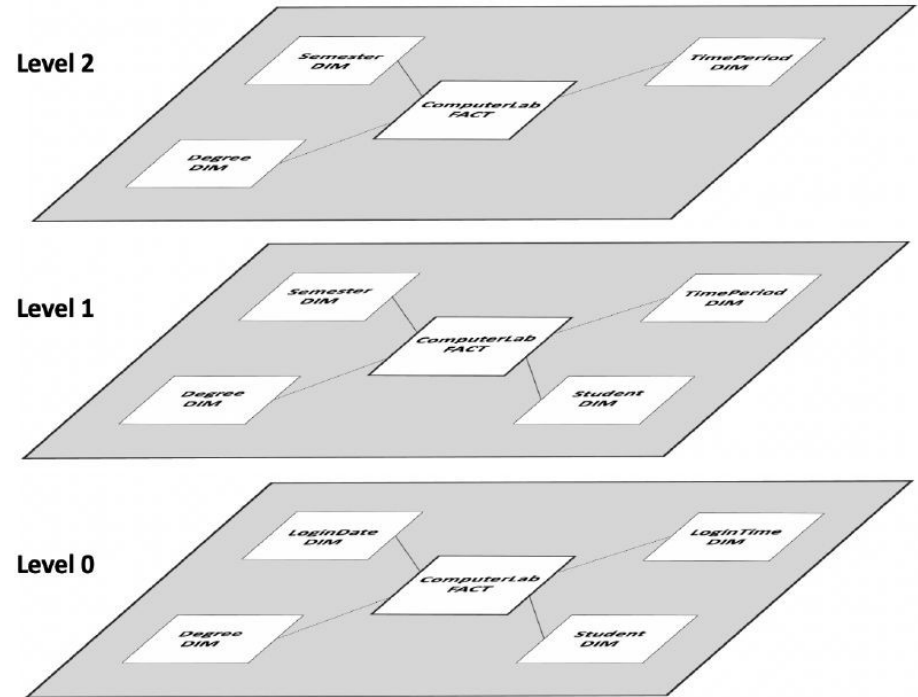


Granularity Reduction In Data Streams

- Group By and Aggregation

Granularity

- **Granularity** is the level of detail at which **data** are stored in a database.
- level-0, the bottom level indicating no aggregation.
- level-1 and level-2 with more aggregation.



Quiz 13

In a sales scenario, if one record is a one-month sales amount, a window of 6 months is used to calculate the running 6-month average sales amount. In this case, the window size is 6 records, and the slide is every record. The number of records in the moving average will be the same as the original number of records. If one year has 12 records of sales, the moving average will also contain 12 records. Hence, no reduction in terms of the number of records. This is a pure moving average (also known as rolling mean).

The above-mentioned case is an example of:

- A. Overlapped Windows - No granularity reduction**
- B. Overlapped Windows - With granularity reduction
- C. Non-Overlapped Windows - Granularity reduction

Mixed Levels of Granularity

- Different levels of granularity combined into one level.
- Mixed level of granularity can be two types:
 - Temporal-based Mixed Levels of Granularity
 - Time based.
 - Spatial-based Mixed Levels of Granularity
 - Space or location based.

Sensor Arrays

- A sensor array is a group of sensors, usually deployed in a certain geometry pattern.
- A network of distributed sensors.
- They add new dimension to the observation, and hence it helps to estimate more parameters, to have better picture of the environment being observed, and improve accuracy.
- Two categories:
 1. Multiple sensors measuring the same things, and
 2. Multiple sensors measuring different things, but they are grouped together.

Sensor Arrays

- Multiple sensors measuring the same things
- Two methods to lower the granularity of sensor arrays that measure the same thing:
 - Method 1: Reduce and then Merge
 - Method 2: Merge and then Reduce

Sensor Arrays

- **Multiple sensors measuring different things**
 - Sensors arrays can be a collection of sensors measuring different things within the same environment.
- Example: A simple indoor sensor array, containing three sensors: air quality, temperature, and humidity.
- **Two methods to lower the granularity of sensor arrays that measure the different thing:**
 - Method 1: Reduce, Normalize, and then Merge
 - Method 2: Normalize, Merge and then Reduce

Unit Summary

1. **Volume** □ Sessions 1, 2, 3, 4
 - How to process Big Data Volume?
2. **Complexity** □ Sessions 5, 6, 7, 8
 - How to apply machine learning algorithms to every aspect of Big Data?
3. **Velocity** □ Sessions 9, 10, 11
 - How to handle and process Fast Streaming Data?

Thank You

Questions?